

- Hitachi Digital — AWS Infrastructure Architect Interview Q&A
 - Table of Contents
 - Section 1: AWS Core Architecture & Design Principles {#section-1}
 - Q1. How do you approach designing a scalable and highly available AWS architecture for a new application?
 - Q2. What is the AWS Well-Architected Framework and how have you applied it in practice?
 - Q3. How do you design for multi-AZ high availability in AWS? What are the trade-offs?
 - Q4. Explain the difference between horizontal and vertical scaling in AWS. When do you use each?
 - Q5. How do you design a serverless architecture on AWS? What are the operational considerations?
 - Q6. How do you design an event-driven architecture on AWS?
 - Q7. What is a multi-region architecture in AWS and when is it justified?
 - Q8. How do you evaluate a new AWS service for adoption in your organization?
 - Section 2: Networking & VPC Design {#section-2}
 - Q9. Design a production VPC architecture from scratch. What are your design decisions?
 - Q10. How do you design security groups and NACLs? What's the difference?
 - Q11. Explain VPC Peering vs. Transit Gateway vs. PrivateLink. When do you use each?
 - Q12. How do you architect connectivity between on-premises and AWS?
 - Q13. How do you handle DNS in AWS for a hybrid environment?
 - Q14. How do you troubleshoot network connectivity issues in AWS?
 - Section 3: Compute — EC2, Lambda & Containers {#section-3}
 - Q15. How do you select the right EC2 instance type for a workload?
 - Q16. How do you implement Auto Scaling on AWS? What are the key configuration parameters?
 - Q17. What is the difference between ECS and EKS? When do you choose each?
 - Q18. How does AWS Lambda work under the hood? Explain cold starts and how to mitigate them.
 - Q19. How do you implement container image security best practices for AWS?

- Q20. How do you use AWS Systems Manager (SSM) for instance management without SSH?
- Section 4: Storage, Databases & Data Services {#section-4}
 - Q21. Compare S3 storage classes and explain when you use each.
 - Q22. How do you choose between RDS, Aurora, and DynamoDB?
 - Q23. How do you implement database backups and disaster recovery for RDS?
 - Q24. How do you use ElastiCache for performance optimization?
 - Q25. How do you design a data lake on AWS?
 - Q26. How do you implement backup and data protection for critical AWS workloads?
- Section 5: IAM, Security & Compliance {#section-5}
 - Q27. Explain IAM roles, policies, and best practices for an enterprise AWS environment.
 - Q28. How do you manage secrets in AWS? Compare options.
 - Q29. How do you implement encryption at rest and in transit for AWS services?
 - Q30. How do you implement AWS GuardDuty and Security Hub for threat detection?
 - Q31. How do you implement AWS Config for compliance monitoring?
- Section 6: Infrastructure-as-Code — Terraform & CloudFormation {#section-6}
 - Q32. How do you structure a Terraform project for AWS infrastructure at scale?
 - Q33. How do you handle sensitive values (passwords, keys) in Terraform?
 - Q34. Compare Terraform with AWS CloudFormation and AWS CDK.
 - Q35. How do you implement a Terraform CI/CD pipeline?
- Section 7: CI/CD, Automation & DevOps Practices {#section-7}
 - Q36. Design a CI/CD pipeline for an application deployed to AWS. Include security scanning.
 - Q37. How do you use AWS CodePipeline and CodeBuild for CI/CD?
 - Q38. How do you implement automation scripts for AWS infrastructure provisioning?
 - Q39. How do you perform zero-downtime deployments on AWS?
- Section 8: Cost Optimization & FinOps {#section-8}
 - Q40. How do you reduce AWS infrastructure costs by 30-40%?
 - Q41. How do you implement AWS cost allocation and tagging strategy?
- Section 9: Reliability, DR & High Availability {#section-9}
 - Q42. How do you design a disaster recovery strategy on AWS?

- [Section 10: Monitoring, Observability & Troubleshooting {#section-10}](#)
 - [Q43. How do you set up comprehensive monitoring for AWS infrastructure?](#)
 - [Q44. How do you use AWS X-Ray for distributed tracing?](#)
 - [Q45. How do you troubleshoot a sudden increase in AWS costs?](#)
- [Section 11: Scenario-Based & Behavioral Questions {#section-11}](#)
 - [Q46. Scenario: Your production RDS database is at 95% CPU. Walk through how you diagnose and resolve it.](#)
 - [Q47. Scenario: You need to migrate a 5TB on-premises database to Aurora PostgreSQL with < 30 minutes downtime. How do you do it?](#)
 - [Q48. Scenario: You're asked to reduce the AWS bill by 30% without impacting availability. What's your plan?](#)
 - [Q49. How do you maintain AWS infrastructure documentation?](#)
 - [Q50. How do you handle a failed Terraform apply in production?](#)

Hitachi Digital — AWS Infrastructure Architect Interview Q&A

Role: AWS Infrastructure Architect | **Company:** Hitachi Digital

Focus: AWS Cloud Architecture, Infrastructure Design, Security, Cost Optimization, IaC, DevOps

Level: Senior / Lead Architect

Table of Contents

- [Section 1: AWS Core Architecture & Design Principles \(Q1–Q8\)](#)
- [Section 2: Networking & VPC Design \(Q9–Q14\)](#)
- [Section 3: Compute — EC2, Lambda & Containers \(Q15–Q20\)](#)
- [Section 4: Storage, Databases & Data Services \(Q21–Q26\)](#)
- [Section 5: IAM, Security & Compliance \(Q27–Q33\)](#)
- [Section 6: Infrastructure-as-Code — Terraform & CloudFormation \(Q34–Q40\)](#)
- [Section 7: CI/CD, Automation & DevOps Practices \(Q41–Q46\)](#)
- [Section 8: Cost Optimization & FinOps \(Q47–Q51\)](#)
- [Section 9: Reliability, DR & High Availability \(Q52–Q57\)](#)
- [Section 10: Monitoring, Observability & Troubleshooting \(Q58–Q63\)](#)

Section 1: AWS Core Architecture & Design Principles {#section-1}

Q1. How do you approach designing a scalable and highly available AWS architecture for a new application?

Answer:

I follow the **AWS Well-Architected Framework** as the baseline, organized around six pillars:

Pillar	Key Decisions
Operational Excellence	IaC for all infra, CI/CD pipelines, runbooks, observability
Security	Least-privilege IAM, encryption at rest/transit, VPC isolation, WAF
Reliability	Multi-AZ, auto-scaling, health checks, graceful degradation
Performance Efficiency	Right-size instances, caching (ElastiCache), CDN (CloudFront), async patterns
Cost Optimization	Reserved/Savings Plans, Spot for batch, rightsizing, tagging
Sustainability	Graviton instances, Spot usage, right-sizing

My design approach:

Step 1: Understand non-functional requirements

- Availability SLA: 99.9% or 99.99%?
- RTO/RPO: How fast must we recover? How much data can we lose?
- Traffic pattern: Constant? Bursty? Seasonal peaks?
- Compliance: HIPAA, PCI, SOC2?

Step 2: Choose the right compute model

EC2 → Long-running stateful workloads, legacy lift-and-shift

ECS/EKS → Containerized microservices, portability

Lambda → Event-driven, short-lived, irregular traffic

Fargate → Containers without node management

Step 3: Design for failure

- No single points of failure (multi-AZ, multi-region for critical)

- Circuit breakers + retries with exponential backoff

- Independent failure domains (separate accounts per env)

Step 4: Define the data tier

Aurora Multi-AZ for relational; DynamoDB for scale; ElastiCache for hot reads

Step 5: Design connectivity

- Public-facing: ALB in public subnets; EC2/Lambda in private subnets

- All internal APIs: Private endpoints or PrivateLink

Reference 3-tier architecture:

Internet



Route 53 (DNS + health routing)



CloudFront (CDN + WAF)



Application Load Balancer (public subnets, multi-AZ)



Auto Scaling Group / ECS Service (private subnets)



Amazon RDS Aurora (private subnets, Multi-AZ)



Amazon ElastiCache (read caching)

Q2. What is the AWS Well-Architected Framework and how have you applied it in practice?

Answer:

The Well-Architected Framework is AWS's set of best practices for building cloud workloads. It consists of **six pillars** and is operationalized through the **Well-Architected Tool** in the console which asks ~200 questions and produces a risk report.

How I apply it in practice:

1. Architecture reviews at project start:

- Run a Well-Architected Review before the first production deployment
- Identify HIGH risk items; create JIRA tickets with owners
- Prioritize: Security HIGH risks → fix immediately; Cost MEDIUM → fix within sprint

2. Automated checks (continuous):

```
# AWS Trusted Advisor: automated checks against Well-Architected principles
aws support describe-trusted-advisor-checks --language en

# AWS Security Hub: aggregates security findings across pillars
aws securityhub get-findings \
  --filters '{"SeverityLabel":[{"Value":"CRITICAL","Comparison":"EQUALS"}]}'
```

3. Real example — Reliability pillar:

On a previous project, a Well-Architected review flagged that our RDS database had no read replica and the application retried failed DB calls with no backoff (tight loop).

Fixes:

- Added RDS read replica; routed read-heavy queries to replica
- Implemented exponential backoff with jitter in the application
- Added CloudWatch alarm for DB connection count approaching max
- Result: Eliminated 3 production incidents per month related to DB overload

Q3. How do you design for multi-AZ high availability in AWS? What are the trade-offs?

Answer:

Multi-AZ principle: Deploy workload components across at least 2 (ideally 3) Availability Zones so that a single AZ failure causes no user-visible outage.

Service-by-service HA patterns:

Service	HA Configuration
EC2	Auto Scaling Group spanning 3 AZs; min 1 instance per AZ
RDS	Multi-AZ deployment; standby in different AZ; ~60s failover
Aurora	Aurora Multi-AZ; up to 15 read replicas across AZs; < 30s failover
ALB	Inherently multi-AZ; route to only healthy AZ targets
ElastiCache	Multi-AZ with automatic failover (Redis cluster mode)
EFS	Regional service; automatically replicated across AZs
Lambda	Inherently multi-AZ; AWS manages placement
SQS	Regional service; inherently replicated across AZs

Trade-offs I always surface:

Trade-off	Impact
Cost	Multi-AZ roughly doubles infrastructure cost for stateful services (RDS standby billing)
Data transfer	Cross-AZ traffic costs \$0.01/GB in each direction — can add up with chatty services
Complexity	More complex to test, debug; latency slightly higher across AZs
Consistency	Synchronous multi-AZ replication (RDS) adds write latency; asynchronous read replicas can serve stale data

My recommendation: Multi-AZ for all production stateful services (RDS, ElastiCache). For development, single-AZ is fine. The cost of downtime for a production database greatly exceeds the multi-AZ standby cost.

Q4. Explain the difference between horizontal and vertical scaling in AWS. When do you use each?

Answer:

Aspect	Vertical Scaling (Scale Up)	Horizontal Scaling (Scale Out)
What it means	Increase instance size (t3.medium → t3.large)	Add more instances behind a load balancer
AWS mechanism	Stop → change instance type → start	Auto Scaling Group; ECS Task count; Lambda concurrency
Downtime	Yes (stop/start required, brief)	No (rolling updates)
Limits	Bounded by largest instance type	Practically unlimited
Cost	Predictable; single bill	Linear with instance count; Spot can optimize
Complexity	Simple	Requires stateless architecture

When I use vertical scaling:

- Databases (RDS) — easier to scale up than shard; minimal application change
- Legacy apps that cannot be made stateless
- Quick fix for urgent capacity crisis (scale up now, architect horizontally later)
- Single-threaded applications that benefit from faster CPU

When I use horizontal scaling:

- Application tier (web servers, API servers) — stateless by design
- Batch processing (more workers = faster throughput)
- Lambda (auto-scales horizontally by default)
- ECS/EKS services

Best practice: Design the application tier to be stateless from day one (sessions in ElastiCache, uploads to S3, no local state). This enables frictionless horizontal scaling.

Q5. How do you design a serverless architecture on AWS? What are the operational considerations?

Answer:

Serverless stack on AWS:

```
API Gateway (HTTP API) → Lambda → DynamoDB / Aurora Serverless
    → SQS → Lambda (async workers)
    → EventBridge → Lambda (event-driven)
    → S3 Event → Lambda (file processing)
```

Lambda design best practices:

```
# Good Lambda function: Single responsibility, fast cold start, idempotent
import boto3
import os
from aws_lambda_powertools import Logger, Tracer, Metrics
from aws_lambda_powertools.metrics import MetricUnit

logger = Logger()
tracer = Tracer()
metrics = Metrics(namespace="OrderService")

# Initialize clients OUTSIDE the handler (reused across warm invocations)
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table(os.environ['ORDERS_TABLE'])

@logger.inject_lambda_context
@tracer.capture_lambda_handler
@metrics.log_metrics
def handler(event, context):
    order_id = event['pathParameters']['orderId']

    # Idempotent: same input always produces same result
    response = table.get_item(Key={'orderId': order_id})

    if 'Item' not in response:
        return {'statusCode': 404, 'body': 'Order not found'}

    metrics.add_metric(name="OrdersFetched", unit=MetricUnit.Count, value=1)
    return {'statusCode': 200, 'body': str(response['Item'])}
```

Operational considerations I always address:

Concern	Solution
Cold start latency	Provisioned Concurrency for latency-sensitive paths; keep functions warm
Timeout configuration	Set timeout based on p99 execution time + buffer; never use max 15min unless needed

Concern	Solution
Concurrency limits	Set Reserved Concurrency to protect downstream services from being overwhelmed
Dead Letter Queues	All async Lambda invocations need a DLQ (SQS/SNS) — never lose failed events
Error handling	Idempotency tokens for state-changing operations; handle partial failures
Observability	Lambda Powertools: structured logging, X-Ray tracing, custom metrics
Dependency management	Lambda Layers for shared libraries; keep deployment package < 50MB
VPC placement	Only put Lambda in VPC if it needs to access VPC resources; adds cold start latency

When NOT to use Lambda:

- Long-running jobs > 15 minutes (use ECS Fargate or Batch)
- High-frequency, small jobs where startup overhead dominates (use ECS)
- Workloads requiring GPU (use EC2/ECS with GPU instances)

Q6. How do you design an event-driven architecture on AWS?

Answer:

Core AWS event-driven services:

Service	Best For
EventBridge	Rule-based routing, AWS service events, SaaS integrations, custom events
SQS	Decoupling, buffering, fan-out with SNS, at-least-once processing
SNS	Fan-out to multiple subscribers (SQS + Lambda + email)

Service	Best For
Kinesis Data Streams	High-throughput real-time streaming, ordering guarantees within shard
MSK (Kafka)	Complex streaming topologies, consumer groups, long retention
S3 Events	Trigger processing on file upload/delete

Event-driven pattern — Order Processing:

```

Customer places order
    |
    ▼
API Gateway → Lambda (OrderService)
    | Publishes OrderPlaced event to EventBridge
    ▼
EventBridge Bus
    ├── Rule: OrderPlaced → SQS → Lambda (InventoryService)
    ├── Rule: OrderPlaced → SQS → Lambda (EmailNotification)
    ├── Rule: OrderPlaced → SQS → Lambda (FraudCheck)
    └── Rule: OrderPlaced → Kinesis → Analytics pipeline
  
```

Each service:

- Processes independently
- Has its own SQS queue (backpressure + retry)
- Has its own DLQ (failed events don't block the queue)
- Is idempotent (EventBridge can deliver at-least-once)

Idempotency implementation (DynamoDB-based):

```

import hashlib
import boto3
from datetime import datetime, timedelta

def process_order_idempotently(event_id: str, order_data: dict):
    dynamodb = boto3.resource('dynamodb')
    idempotency_table = dynamodb.Table('IdempotencyKeys')

    try:
        # Conditional write: fails if key already exists
        idempotency_table.put_item(
            Item={
                'eventId': event_id,
                'processedAt': datetime.utcnow().isoformat(),
                'ttl': int((datetime.utcnow() +
timedelta(days=7)).timestamp())
            },
            ConditionExpression='attribute_not_exists(eventId)'
        )
    
```

```
)
except dynamodb.meta.client.exceptions.ConditionalCheckFailedException:
    print(f"Duplicate event {event_id}, skipping")
    return # Already processed

# Process the order (only reaches here if first time)
process_order(order_data)
```

Q7. What is a multi-region architecture in AWS and when is it justified?

Answer:

Multi-region means deploying workload components in two or more AWS regions (e.g., us-east-1 + eu-west-1).

When it's justified:

Use Case	Justification
Latency optimization	Serve users from the nearest region (< 50ms vs > 150ms)
Data residency / sovereignty	EU user data must stay in EU (GDPR)
Disaster recovery	RTO < 15 minutes (active-passive); or zero downtime (active-active)
Compliance	Regulator requires geographic redundancy (financial services, healthcare)
Business continuity	AWS region-level outage (rare but happened: us-east-1 Dec 2021)

When it's NOT justified:

- Most applications with RTO/RPO requirements > 1 hour → Multi-AZ is sufficient
- Small teams that cannot operate multi-region
- Cost is a primary constraint (doubles infrastructure cost)

Multi-region patterns:

Pattern 1: Active-Passive (Pilot Light)
Primary: us-east-1 (all traffic)
Secondary: eu-west-1 (core infra running, minimal capacity)
Failover: Route 53 health check → failover record → eu-west-1
RTO: 15–30 minutes; RPO: ~1 minute (Aurora Global DB lag)
Cost: +30–40% vs single region

Pattern 2: Active-Active (Warm Standby)
us-east-1: 50% traffic
eu-west-1: 50% traffic
Route 53 latency-based routing
Aurora Global DB: Primary in us-east-1, replica in eu-west-1
RTO: < 1 minute (traffic re-routing); RPO: < 1 second
Cost: ~2x single region

Aurora Global Database for multi-region:

```
resource "aws_rds_global_cluster" "orders" {
  global_cluster_identifier = "orders-global"
  engine                    = "aurora-postgresql"
  engine_version            = "15.4"
  database_name              = "orders"
  storage_encrypted          = true
}

resource "aws_rds_cluster" "primary" {
  provider           = aws.us_east_1
  cluster_identifier = "orders-primary"
  global_cluster_identifier = aws_rds_global_cluster.orders.id
  engine              = "aurora-postgresql"
  # ... typical RPO: < 1 second replication lag
}
```

Q8. How do you evaluate a new AWS service for adoption in your organization?

Answer:

My evaluation framework:

1. Functional fit:

- Does it solve the problem better than what we have?
- What specific capability does it add?

- What's the feature gap vs. what we need?

2. Operational readiness:

- Is it Generally Available (GA) or Preview? (Avoid Preview for production)
- What's the SLA? (Most AWS managed services: 99.9% or 99.99%)
- Does it integrate with our existing tooling (Terraform provider, CloudFormation support)?
- Is there a managed upgrade path?

3. Security and compliance:

- Is it within scope for our compliance certifications? (PCI, HIPAA, SOC2)
- Check: aws.amazon.com/compliance/services-in-scope
- Does it support encryption at rest with KMS CMK?
- Does it support VPC endpoints (private connectivity)?

4. Cost model:

- Pricing model: per-request, per-GB, per-hour?
- Total Cost of Ownership vs. self-managed alternative
- Are there free tier benefits? Reserved pricing available?

5. Proof of Concept:

- Build a time-boxed PoC (1-2 weeks) in a sandbox account
- Measure: performance, cost, operational effort
- Involve the team that will own it day-to-day

6. Build vs. Buy decision:

Buy (use the managed service) when:

- AWS manages the undifferentiated heavy lifting (patching, HA, scaling)
- Total cost of self-managing > service cost
- Team lacks expertise to self-manage reliably

Build/Self-host when:

- Specific customization needed that managed service doesn't support
 - Compliance requirement that managed service doesn't meet
 - Cost difference is extreme and team has expertise
-

Section 2: Networking & VPC Design

{#section-2}

Q9. Design a production VPC architecture from scratch. What are your design decisions?

Answer:

Production VPC design:

VPC: 10.0.0.0/16 (65,534 usable IPs)		
AZ-1 (us-east-1a)	AZ-2 (us-east-1b)	AZ-3 (us-east-1c)
Public Subnet	Public Subnet	Public Subnet
10.0.1.0/24	10.0.2.0/24	10.0.3.0/24
(ALB, NAT Gateway)	(ALB, NAT Gateway)	(ALB, NAT Gateway)
Private App Subnet	Private App Subnet	Private App Subnet
10.0.11.0/24	10.0.12.0/24	10.0.13.0/24
(EC2, ECS, Lambda)	(EC2, ECS, Lambda)	(EC2, ECS, Lambda)
Private DB Subnet	Private DB Subnet	Private DB Subnet
10.0.21.0/24	10.0.22.0/24	10.0.23.0/24
(RDS, ElastiCache)	(RDS, ElastiCache)	(RDS, ElastiCache)

Key design decisions:

1. CIDR planning — leave room to grow:

```
/16 VPC = 65K IPs (enough for large enterprise)
/24 subnets = 251 usable IPs per subnet
/20 subnets if you anticipate large EKS clusters (EKS uses 1 IP per pod)
Reserve /8 blocks for future VPC peering (avoid CIDR overlap)
```

2. Three-tier subnet model:

- **Public subnets:** Internet-facing resources only (ALB, NAT Gateway, bastion/SSM)
- **Private app subnets:** All compute (EC2, ECS, Lambda, EKS nodes)
- **Private DB subnets:** Data layer only; no compute allowed here

3. Internet access from private subnets — NAT Gateway:

```
# One NAT Gateway per AZ (avoid cross-AZ NAT traffic charges)
resource "aws_nat_gateway" "az1" {
  allocation_id = aws_eip.nat_az1.id
  subnet_id     = aws_subnet.public_az1.id
}

# Private subnet route: all internet traffic via local NAT GW
resource "aws_route" "private_internet_az1" {
  route_table_id      = aws_route_table.private_az1.id
  destination_cidr_block = "0.0.0.0/0"
  nat_gateway_id      = aws_nat_gateway.az1.id
}
```

4. VPC Flow Logs enabled for all traffic:

```
resource "aws_flow_log" "vpc" {
  vpc_id            = aws_vpc.main.id
  traffic_type      = "ALL"
  iam_role_arn      = aws_iam_role.flow_log.arn
  log_destination   = aws_cloudwatch_log_group.vpc_flow_logs.arn
}
```

5. VPC Endpoints for AWS services (avoid NAT Gateway costs + latency):

```
# S3 Gateway Endpoint (free; avoids NAT Gateway for S3 traffic)
resource "aws_vpc_endpoint" "s3" {
  vpc_id            = aws_vpc.main.id
  service_name      = "com.amazonaws.us-east-1.s3"
  vpc_endpoint_type = "Gateway"
}
```



```
route_table_ids = [aws_route_table.private_az1.id]
}

# Interface Endpoints for SSM, Secrets Manager, ECR (no internet needed)
resource "aws_vpc_endpoint" "ssm" {
  vpc_id            = aws_vpc.main.id
  service_name      = "com.amazonaws.us-east-1.ssm"
  vpc_endpoint_type = "Interface"
  subnet_ids       = aws_subnet.private_app[*].id
  security_group_ids = [aws_security_group.vpc_endpoints.id]
  private_dns_enabled = true
}
```

Q10. How do you design security groups and NACLs? What's the difference?

Answer:

Key differences:

Feature	Security Group	NACL
Level	Instance/ENI level	Subnet level
State	Stateful (return traffic auto-allowed)	Stateless (must allow both directions)
Rules	Allow only	Allow + Deny
Rule evaluation	All rules evaluated	Rules evaluated in order (lowest number wins)
Default	Deny all inbound; Allow all outbound	Allow all in/out
Use case	Primary defense; fine-grained control	Subnet-level broad controls; additional layer

Security group design principles:

```
# ALB Security Group
resource "aws_security_group" "alb" {
  name = "alb-sg"
  vpc_id = aws_vpc.main.id
}
```

```

ingress {
  description = "HTTPS from internet"
  from_port   = 443
  to_port     = 443
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

ingress {
  description = "HTTP redirect from internet"
  from_port   = 80
  to_port     = 80
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

egress {
  description      = "To app tier only (not internet)"
  from_port        = 8080
  to_port          = 8080
  protocol         = "tcp"
  security_groups  = [aws_security_group.app.id]
}
}

# App Tier Security Group – reference ALB SG, not CIDR
resource "aws_security_group" "app" {
  name     = "app-sg"
  vpc_id   = aws_vpc.main.id

  ingress {
    description      = "From ALB only"
    from_port        = 8080
    to_port          = 8080
    protocol         = "tcp"
    security_groups  = [aws_security_group.alb.id] # Not a CIDR range
  }

  egress {
    description      = "To DB tier"
    from_port        = 5432
    to_port          = 5432
    protocol         = "tcp"
    security_groups  = [aws_security_group.db.id]
  }
}

# DB Tier Security Group
resource "aws_security_group" "db" {
  name     = "db-sg"
  vpc_id   = aws_vpc.main.id

  ingress {
    description      = "From app tier only"
    from_port        = 5432
    to_port          = 5432
    protocol         = "tcp"
  }
}

```

```

    security_groups = [aws_security_group.app.id]
}
# No egress needed; DB doesn't initiate connections
}

```

Critical rule: Always reference security group IDs, not CIDR ranges, for internal traffic. If you use CIDR ranges, adding a new subnet requires updating security group rules.

NACL usage: I use NACLs sparingly — mainly to:

- Block known malicious IP ranges (quick blocklist)
- Deny specific subnet-to-subnet traffic categories as a second layer
- For PCI-DSS: NACL on the CDE subnet to restrict outbound destinations

Q11. Explain VPC Peering vs. Transit Gateway vs. PrivateLink. When do you use each?

Answer:

Feature	VPC Peering	Transit Gateway	PrivateLink
Model	1:1 direct connection	Hub-and-spoke (N:M)	Service endpoint
Transitive routing	✗ No	✓ Yes	N/A
Scale	~125 connections/VPC	5,000+ attachments	Millions of consumers
Cost	Free + data transfer	0.05/hr/attachment+ 0.02/GB	0.01/hr+ 0.01/GB
Cross-account	✓ (manual accept)	✓	✓
Cross-region	✓	✓ (TGW peering)	✗ (same region)

Decision guide:

```

≤ 5 VPCs, simple setup?
→ VPC Peering (no hub cost; straightforward)

```

- > 5 VPCs or need transitive routing?
 - Transit Gateway (scales; route tables give fine-grained control)
- Exposing a specific service/API to another account/VPC securely?
 - PrivateLink (service consumer gets no network access; only service access)
 - Use case: SaaS providers, shared internal services, vendor integrations

Practical example — PrivateLink for internal payment service:

```
# Provider account: expose payment service via NLB
resource "aws_lb" "payments" {
  name            = "payments-nlb"
  load_balancer_type = "network"
  internal        = true
  subnets        = var.private_subnet_ids
}

resource "aws_vpc_endpoint_service" "payments" {
  acceptance_required = false
  network_load_balancer_arns = [aws_lb.payments.arn]

  allowed_principals = ["arn:aws:iam::${var.consumer_account_id}:root"]
}

# Consumer account: create VPC endpoint
resource "aws_vpc_endpoint" "payments" {
  service_name      = "com.amazonaws.vpce.us-east-1.vpce-svc-xxx"
  vpc_id            = var.vpc_id
  subnet_ids        = var.private_subnet_ids
  security_group_ids = [aws_security_group.payments_endpoint.id]
  vpc_endpoint_type = "Interface"
  private_dns_enabled = true
}
```

Q12. How do you architect connectivity between on-premises and AWS?

Answer:

Two primary options:

AWS Direct Connect (dedicated):

On-premises → Direct Connect location → AWS backbone → VPC

- Dedicated 1Gbps or 10Gbps bandwidth
- Consistent, low latency (< 10ms for most regions)
- Private BGP peering: access VPC resources privately
- Public BGP peering: access S3, DynamoDB, other AWS public endpoints
- Cost: port charge (~\$0.30/hr) + data transfer (< \$0.02/GB outbound)
- SLA: 99.9% (single connection); 99.99% (redundant connections)

AWS Site-to-Site VPN (IPsec over internet):

On-premises VPN appliance → Internet → AWS VPN endpoint → VPC

- Encrypted IPsec tunnel (2 tunnels per connection for redundancy)
- Variable latency (internet-dependent)
- Max 1.25 Gbps per tunnel
- Cost: \$0.05/hr per VPN connection + data transfer
- Setup time: hours (vs. weeks for Direct Connect)

My recommended hybrid connectivity architecture:

Production: AWS Direct Connect (primary) + VPN (failover)

- Direct Connect: for predictable, high-bandwidth production traffic
- VPN as backup: automatically fails over via BGP route preference

Development/Staging: VPN only (cost-effective)

- VPN sufficient for developer access and lower traffic volumes

Implementation:

- Virtual Private Gateway (VGW) attached to production VPC
- Transit Gateway for connecting multiple VPCs to Direct Connect
- Direct Connect Gateway: connects one Direct Connect to VPCs in multiple regions

Q13. How do you handle DNS in AWS for a hybrid environment?

Answer:

Route 53 Resolver for hybrid DNS:

On-premises DNS (e.g., Active Directory at corp.company.com)

|

▼ DNS query for *.corp.company.com
Route 53 Resolver Outbound Endpoint (in VPC)
└─ Forwards to on-premises DNS servers (10.0.0.2)

On-premises DNS server
| DNS query for *.aws.internal
▼
Route 53 Resolver Inbound Endpoint (ENI in private subnet)
└─ Resolves against Route 53 Private Hosted Zone

```
# Inbound endpoint (on-prem → AWS)
resource "aws_route53_resolver_endpoint" "inbound" {
  name      = "inbound-from-onprem"
  direction = "INBOUND"
  security_group_ids = [aws_security_group.dns_resolver.id]

  ip_address {
    subnet_id = aws_subnet.private_az1.id
  }
  ip_address {
    subnet_id = aws_subnet.private_az2.id
  }
}

# Outbound endpoint + rule (AWS → on-prem for corp.company.com)
resource "aws_route53_resolver_rule" "corp_dns" {
  domain_name      = "corp.company.com"
  rule_type        = "FORWARD"
  resolver_endpoint_id = aws_route53_resolver_endpoint.outbound.id

  target_ip {
    ip    = "10.10.0.2" # On-premises DNS server
    port = 53
  }
}
```

Q14. How do you troubleshoot network connectivity issues in AWS?

Answer:

Structured troubleshooting approach:

Symptom: Cannot reach EC2 instance on port 443

Step 1: Check Security Groups

- Instance SG: inbound rule allows 443 from source?
- Source SG or CIDR must match

Step 2: Check NACLs

- Subnet NACL: inbound rule allows 443?
- NACL is stateless: outbound ephemeral port (1024–65535) must also be allowed

Step 3: Check Route Tables

- Subnet has route to destination (0.0.0.0/0 → IGW for public; NAT for private)?
- IGW attached to VPC?

Step 4: Check instance state

- Is instance running? Is it passing health checks?
- Does instance have a public IP (if needed)?

Step 5: Check VPC Flow Logs

- Filter: destination-port=443, action=REJECT
- This shows exactly which layer is dropping traffic

Step 6: VPC Reachability Analyzer (graphical tool)

- Source: my computer's public IP
- Destination: EC2 instance
- Shows exact path and where it breaks

AWS CLI reachability analyzer:

```
# Create a network insights path
aws ec2 create-network-insights-path \
  --source eni-source \
  --destination eni-dest \
  --protocol TCP \
  --destination-port 443

# Run analysis
aws ec2 start-network-insights-analysis \
  --network-insights-path-id nip-xxx

# Get results
aws ec2 describe-network-insights-analyses \
  --network-insights-analysis-ids nia-xxx
```

Common issues I've fixed:

Issue	Root Cause	Fix
SSH not working	SG missing port 22 inbound; or using wrong key pair	Add SG rule; use correct .pem

Issue	Root Cause	Fix
App can't reach RDS	SG rule uses CIDR instead of SG ref; CIDR changed	Switch to SG-to-SG reference
Lambda can't reach internet	Lambda in VPC but no NAT Gateway	Add NAT GW or use VPC Endpoint
Inter-AZ latency spike	Cross-AZ traffic via wrong NAT GW	One NAT GW per AZ; route each AZ to local NAT GW

Section 3: Compute — EC2, Lambda & Containers {#section-3}

Q15. How do you select the right EC2 instance type for a workload?

Answer:

Instance family guide:

Family	Optimized For	Examples	Use Case
t3/t4g	Burstable CPU	t3.medium, t4g.large	Dev/test, low-traffic web
m6i/m7i	General purpose	m6i.xlarge, m7i.2xlarge	Web servers, app servers
c6i/c7i	Compute-intensive	c6i.4xlarge	High-CPU batch, encoding
r6i/r7i	Memory-intensive	r6i.4xlarge	In-memory caching, heavy DBs
i3/i4i	Storage-intensive	i3.large, i4i.xlarge	NoSQL DBs, local NVMe
p3/p4d	GPU	p3.2xlarge	ML training
inf2	ML inference	inf2.xlarge	SageMaker inference

Family	Optimized For	Examples	Use Case
g4dn	GPU (general)	g4dn.xlarge	Video rendering, ML inference

Graviton (ARM) instances: 20-40% better price/performance for most workloads.

Always test with arm64 before choosing x86:

- **m7g** (Graviton3) vs **m7i** (Intel): ~20% cheaper for same performance
- Requires arm64-compatible binaries and Docker images

My selection methodology:

1. Identify workload profile:
 - CPU-bound (video encoding) → c family
 - Memory-bound (in-memory cache) → r family
 - Balanced (web app) → m family
 - Burstable (dev, cron jobs) → t family
2. Estimate right size using CloudWatch metrics (existing workload):
 - Target CPU: 40-60% average utilization
 - Target Memory: 50-70% average utilization
 - If consistently < 20% CPU → downsize
3. Test Graviton equivalent:
 - Run parallel load test: m6i.xlarge vs m7g.xlarge
 - ~20-40% cost savings with comparable or better performance
4. Choose purchasing model:
 - Steady workload → Savings Plans (1-year, no upfront)
 - Variable/batch → Spot instances
 - Unpredictable → On-demand initially; analyze after 30 days

Q16. How do you implement Auto Scaling on AWS? What are the key configuration parameters?

Answer:

EC2 Auto Scaling Group key parameters:

```
resource "aws_autoscaling_group" "app" {
  name           = "app-asg"
  min_size       = 2      # Never go below; maintains HA
```

```

max_size           = 20      # Cost ceiling
desired_capacity   = 4      # Starting point

# Spread across AZs for HA
vpc_zone_identifier = aws_subnet.private_app[*].id

# Use launch template (not launch config – deprecated)
launch_template {
  id      = aws_launch_template.app.id
  version = "$Latest"
}

# Health check – use ELB, not EC2 (detects app-level health, not just
instance state)
health_check_type      = "ELB"
health_check_grace_period = 300 # Time for instance to start before
health checks

# Warm pool: pre-warm instances for faster scaling
warm_pool {
  pool_state      = "Stopped" # Stopped = cheaper; Running =
faster scale-out
  min_size        = 1
  max_group_prepared_capacity = 3
}

# Instance refresh: rolling update without downtime
instance_refresh {
  strategy = "Rolling"
  preferences {
    min_healthy_percentage = 90
    instance_warmup        = 300
  }
}
}

```

Scaling policies — types:

Type	Description	When to Use
Target Tracking	Maintain a metric at a target value (e.g., 70% CPU)	Most use cases; simple
Step Scaling	Scale by N instances based on breach magnitude	Granular control needed
Scheduled Scaling	Scale at predefined times	Known traffic patterns (peak hours)
Predictive Scaling	ML-based; proactive scale-out before traffic peaks	Cyclical load patterns

```
# Target tracking: keep CPU at 70%
resource "aws_autoscaling_policy" "cpu_target" {
  autoscaling_group_name = aws_autoscaling_group.app.name
  policy_type             = "TargetTrackingScaling"

  target_tracking_configuration {
    predefined_metric_specification {
      predefined_metric_type = "ASGAverageCPUUtilization"
    }
    target_value      = 70.0
    disable_scale_in = false # Allow scale-in when load drops
  }
}
```

Scale-in protection for stateful instances:

```
# Protect a specific instance from scale-in (e.g., running a critical job)
aws autoscaling set-instance-protection \
  --instance-ids i-abc123 \
  --auto-scaling-group-name app-asg \
  --protected-from-scale-in
```

Q17. What is the difference between ECS and EKS? When do you choose each?

Answer:

Feature	Amazon ECS	Amazon EKS
Orchestration	AWS proprietary	Kubernetes (open-source)
Learning curve	Low (AWS-native concepts)	High (K8s expertise needed)
Managed control plane	Fully managed (free)	Managed (\$0.10/hr/cluster)
Ecosystem	AWS-centric	Large OSS ecosystem (Helm, ArgoCD, operators)
Multi-cloud portability	Low (AWS-specific)	High (K8s is cloud-agnostic)

Feature	Amazon ECS	Amazon EKS
Fargate support	✅ Strong	✅ Supported
Advanced networking	✅ awsipc mode (task-level SG)	✅ VPC CNI (pod-level SG via IRSA)
IAM integration	Task IAM roles	IRSA (IAM Roles for Service Accounts)
Service mesh	AWS App Mesh	Istio / Linkerd / App Mesh

Choose ECS when:

- Team is AWS-focused; no Kubernetes expertise on team
- Simple containerized workloads (< 50 services)
- Want minimal operational overhead
- Using Fargate for serverless containers (no node management)
- AWS-specific integrations are the priority (EventBridge, Step Functions)

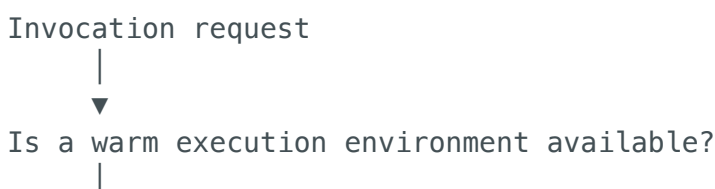
Choose EKS when:

- Kubernetes expertise exists on team
- Multi-cloud portability required (may migrate to Azure/GCP later)
- Large microservices ecosystem (50+ services)
- Advanced GitOps required (ArgoCD, Flux)
- Platform team building self-service developer platform

Q18. How does AWS Lambda work under the hood? Explain cold starts and how to mitigate them.

Answer:

Lambda execution lifecycle:



- └ YES (warm start): Invoke handler directly (~1ms overhead)
- └ NO (cold start):
 1. Download deployment package from S3 (~100ms)
 2. Start execution environment (micro-VM)
 3. Initialize runtime (Python/Node.js) (~100-500ms)
 4. Run initialization code (outside handler) (~depends on code)
 5. Run handler (~application latency)Total cold start overhead: 200ms – 2000ms depending on runtime/size

Cold start factors:

Factor	Impact	Mitigation
Runtime	Java/C# → slowest; Python/Node → fastest	Use Python/Node for latency-sensitive
Package size	Larger = slower download	Keep < 10MB; use Lambda Layers; avoid unnecessary dependencies
VPC placement	+500ms for ENI attachment	Avoid VPC unless necessary; use VPC Endpoints instead
Memory	Low memory → slower CPU → slower init	Min 512MB for Lambda in VPC
Initialization code	Heavy init (DB connect, ML model load) → slow	Cache connections globally; use lazy loading

Mitigation strategies:

```
# Strategy 1: Move expensive initialization OUTSIDE the handler
# Initialized once per execution environment (not per invocation)
import boto3
import os

# This runs during INIT phase, shared across all invocations in this env
db_client = boto3.resource('dynamodb')
table = db_client.Table(os.environ['TABLE_NAME'])

def handler(event, context):
    # db_client reused – no initialization cost here
    return table.get_item(Key={'id': event['id']})
```

```
# Strategy 2: Provisioned Concurrency (pre-warmed environments)
resource "aws_lambda_provisioned_concurrency_config" "payments" {
```

```

function_name                =
aws_lambda_function.payments.function_name
qualifier                    = aws_lambda_alias.live.name
provisioned_concurrent_executions = 10 # 10 pre-warmed instances
# Cost: ~$0.015/GB-hr for provisioned; worthwhile for p99 < 100ms
requirement
}

```

```

# Strategy 3: Lambda SnapStart (Java only) – reduces cold start by ~90%
resource "aws_lambda_function" "java_api" {
  # ...
  snap_start {
    apply_on = "PublishedVersions"
  }
}

```

Q19. How do you implement container image security best practices for AWS?

Answer:

Secure Dockerfile:

```

# 1. Use specific version tags (not latest)
FROM python:3.12.3-slim-bookworm

# 2. Create non-root user
RUN groupadd -r appgroup && useradd -r -g appgroup appuser

# 3. Install only what's needed; clean up in same layer
RUN apt-get update && \
    apt-get install -y --no-install-recommends libpq5 && \
    rm -rf /var/lib/apt/lists/*

WORKDIR /app

# 4. Copy and install dependencies first (layer caching)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 5. Copy application code
COPY --chown=appuser:appgroup src/ .

# 6. Run as non-root
USER appuser

```

```
# 7. No sensitive data in ENV (use Secrets Manager / Parameter Store at runtime)
# BAD: ENV DB_PASSWORD=secret123
# GOOD: Fetch from Secrets Manager in application code

EXPOSE 8080
CMD ["python", "app.py"]
```

ECR image scanning:

```
resource "aws_ecr_repository" "app" {
  name                = "app-service"
  image_tag_mutability = "IMMUTABLE" # Cannot overwrite tags

  image_scanning_configuration {
    scan_on_push = true # Scan every pushed image
  }

  encryption_configuration {
    encryption_type = "KMS"
    kms_key         = aws_kms_key.ecr.arn
  }
}
```

```
# Check scan results before deploying
aws ecr describe-image-scan-findings \
  --repository-name app-service \
  --image-id imageTag=abc123sha \
  --query 'imageScanFindings.findingSeverityCounts'
# Block deployment if CRITICAL findings exist
```

ECR lifecycle policy (reduce storage costs + attack surface):

```
{
  "rules": [
    {
      "rulePriority": 1,
      "description": "Remove untagged images after 7 days",
      "selection": {
        "tagStatus": "untagged",
        "countType": "sinceImagePushed",
        "countUnit": "days",
        "countNumber": 7
      },
      "action": { "type": "expire" }
    }
  ]
}
```

```
]
}
```

Q20. How do you use AWS Systems Manager (SSM) for instance management without SSH?

Answer:

Why SSM over SSH:

- No inbound port 22 (eliminates SSH attack surface entirely)
- No key pair management
- Full session audit trail in CloudTrail
- Patch management, run commands, inventory — all centralized
- Works without internet access via VPC Endpoint

SSM Session Manager (interactive shell):

```
# Connect to instance without SSH
aws ssm start-session --target i-abc123

# Port forwarding (access RDS without SSH tunnel)
aws ssm start-session \
  --target i-abc123 \
  --document-name AWS-StartPortForwardingSessionToRemoteHost \
  --parameters '{"host":["rds.example.com"],"portNumber":
["5432"],"localPortNumber":["5432"]}'
```

IAM role for SSM on EC2:

```
resource "aws_iam_role_policy_attachment" "ssm" {
  role      = aws_iam_role.ec2_instance.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore"
  # This is all that's needed – no manual agent installation on Amazon Linux
  2023
}
```

SSM Run Command (automation without login):


```
# Run a command on multiple instances by tag
aws ssm send-command \
  --targets "Key=tag:Environment,Values=production" \
  --document-name "AWS-RunShellScript" \
  --parameters commands=["sudo systemctl restart nginx"] \
  --output-s3-bucket-name my-ssm-logs
```

Patch Manager (automated OS patching):

```
resource "aws_ssm_patch_baseline" "amazon_linux" {
  name            = "amazon-linux-2023-baseline"
  operating_system = "AMAZON_LINUX_2023"

  approval_rule {
    approve_after_days = 7 # Auto-approve patches 7 days after release
    compliance_level   = "HIGH"

    patch_filter {
      key    = "CLASSIFICATION"
      values = ["Security", "Bugfix"]
    }

    patch_filter {
      key    = "SEVERITY"
      values = ["Critical", "Important"]
    }
  }
}
```

Section 4: Storage, Databases & Data Services {#section-4}

Q21. Compare S3 storage classes and explain when you use each.

Answer:

Storage Class	Retrieval	Min Duration	Cost/GB-month	Use Case
S3 Standard	Instant	None	\$0.023	Active data, frequent access
S3 Standard-IA	Instant	30 days	\$0.0125	Infrequent access, backups (30+ day retention)
S3 One Zone-IA	Instant	30 days	\$0.01	Reproducible data; less critical (no multi-AZ)
S3 Glacier Instant	Instant	90 days	\$0.004	Archive needing immediate retrieval
S3 Glacier Flexible	1-12 hours	90 days	\$0.0036	Long-term archive, non-urgent retrieval
S3 Glacier Deep Archive	12-48 hours	180 days	\$0.00099	Compliance archives, 7+ year retention
S3 Intelligent-Tiering	Instant/delayed	None	\$0.023 (active)	Unpredictable access patterns

Lifecycle policy (automate transitions):

```
resource "aws_s3_bucket_lifecycle_configuration" "app_logs" {
  bucket = aws_s3_bucket.app_logs.id

  rule {
    id      = "log-archival"
    status  = "Enabled"

    transition {
      days          = 30
      storage_class = "STANDARD_IA"    # After 30 days
    }

    transition {
      days          = 90
      storage_class = "GLACIER_INSTANT_RETRIEVAL" # After 90 days
    }

    expiration {
      days = 365 # Delete after 1 year
    }
  }
}
```

```
}  
}
```

S3 security best practices:

```
# Block all public access at the account level  
resource "aws_s3_account_public_access_block" "default" {  
  block_public_acls      = true  
  block_public_policy    = true  
  ignore_public_acls    = true  
  restrict_public_buckets = true  
}  
  
# Enforce SSL-only access  
resource "aws_s3_bucket_policy" "enforce_ssl" {  
  bucket = aws_s3_bucket.app.id  
  policy = jsonencode({  
    Statement = [{  
      Sid      = "DenyNonSSL"  
      Effect   = "Deny"  
      Principal = "*"   
      Action   = "s3:*"  
      Resource = ["${aws_s3_bucket.app.arn}", "${aws_s3_bucket.app.arn}/*"]  
      Condition = {  
        Bool = { "aws:SecureTransport" = "false" }  
      }  
    }]  
  })  
}
```

Q22. How do you choose between RDS, Aurora, and DynamoDB?

Answer:

Factor	RDS	Aurora	DynamoDB
Data model	Relational (SQL)	Relational (SQL)	NoSQL (key-value/document)
Scale	Vertical (up to 128TB)	Up to 128TB; auto-storage	Virtually unlimited
Throughput	Up to ~80K IOPS	Up to 200K IOPS	Millions of RPS

Factor	RDS	Aurora	DynamoDB
Replication	Synchronous multi-AZ + async replicas	Up to 15 read replicas; < 100ms lag	Global Tables (multi-region)
Failover	~60-120 seconds	< 30 seconds	Automatic (no concept of failover)
Cost model	Instance + storage + I/O	Instance + storage (I/O optimized available)	On-demand or provisioned capacity
Best for	Existing SQL apps, complex queries	New SQL workloads, high availability	Key-value lookups, session store, gaming

Aurora Serverless v2: Scales ACUs (Aurora Capacity Units) in 0.5 ACU increments in seconds. Excellent for variable workloads (dev environments scale to 0; production scales to 128 ACUs).

DynamoDB table design (critical to get right upfront):

```
# Single-table design: store multiple entity types in one table
# PK (partition key) + SK (sort key) enable flexible access patterns

# Access patterns:
# - Get user by ID → PK: USER#{userId}, SK: PROFILE
# - Get user orders → PK: USER#{userId}, SK: ORDER#{orderId}
# - Get order details → PK: ORDER#{orderId}, SK: DETAILS

table = dynamodb.Table('AppData')

# Store user
table.put_item(Item={
    'PK': f'USER#{user_id}',
    'SK': 'PROFILE',
    'name': 'John Doe',
    'email': 'john@example.com',
    'createdAt': '2025-01-01'
})

# Store order
table.put_item(Item={
    'PK': f'USER#{user_id}',
    'SK': f'ORDER#{order_id}',
    'amount': 150.00,
    'status': 'pending'
})
```

```
# Query all orders for a user
response = table.query(
    KeyConditionExpression=Key('PK').eq(f'USER#{user_id}') &
    Key('SK').begins_with('ORDER#')
)
```

Q23. How do you implement database backups and disaster recovery for RDS?

Answer:

RDS backup strategy:

```
resource "aws_db_instance" "production" {
  # Automated backups
  backup_retention_period = 35    # Keep 35 days of automated backups
  (max)
  backup_window           = "03:00-04:00" # UTC; low-traffic window
  delete_automated_backups = false # Retain backups if instance is deleted

  # Maintenance window (minor version upgrades, patching)
  maintenance_window      = "sun:04:00-sun:05:00"
  auto_minor_version_upgrade = true

  # Deletion protection (prevents accidental delete)
  deletion_protection = true

  # Multi-AZ (synchronous standby for HA)
  multi_az = true

  # Enhanced monitoring (per-second OS metrics)
  monitoring_interval = 60
  monitoring_role_arn = aws_iam_role.rds_monitoring.arn

  # Performance Insights (SQL-level metrics)
  performance_insights_enabled      = true
  performance_insights_retention_period = 731 # 2 years

  # Encryption
  storage_encrypted = true
  kms_key_id        = aws_kms_key.rds.arn
}
```

Cross-region backup replication:

```
# Replicate automated snapshots to DR region
resource "aws_db_instance_automated_backups_replication" "dr" {
  source_db_instance_arn = aws_db_instance.production.arn
  retention_period       = 14

  provider = aws.dr_region # us-west-2

  kms_key_id = aws_kms_key.rds_dr.arn # KMS key in DR region
}
```

Point-in-time recovery:

```
# Restore to any second within the retention window
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier production-db \
  --target-db-instance-identifier production-db-restored \
  --restore-time 2025-03-15T14:30:00Z \
  --db-subnet-group-name my-subnet-group \
  --vpc-security-group-ids sg-xxx
```

DR testing cadence: Test recovery quarterly. Document RTO achieved (time to restore from snapshot). Update runbook with actual timings.

Q24. How do you use ElastiCache for performance optimization?

Answer:

Caching patterns:

Pattern	Description	Use Case
Cache-aside (lazy loading)	App checks cache; miss → DB → write to cache	Most flexible; handles cache misses gracefully
Write-through	Write to cache and DB simultaneously	Read-heavy; cache always current
Write-behind	Write to cache; async write to DB	Write-heavy; eventual consistency acceptable

Cache-aside implementation:

```
import redis
import json
import boto3
from functools import wraps

redis_client = redis.Redis(
    host=os.environ['ELASTICACHE_ENDPOINT'],
    port=6379,
    ssl=True, # In-transit encryption
    decode_responses=True
)

def get_user(user_id: str):
    cache_key = f"user:{user_id}"

    # Check cache first
    cached = redis_client.get(cache_key)
    if cached:
        return json.loads(cached)

    # Cache miss: fetch from DB
    dynamodb = boto3.resource('dynamodb')
    table = dynamodb.Table('Users')
    response = table.get_item(Key={'userId': user_id})
    user = response.get('Item')

    if user:
        # Write to cache with TTL (avoid stale data)
        redis_client.setex(cache_key, 3600, json.dumps(user)) # 1 hour TTL

    return user

def invalidate_user_cache(user_id: str):
    # Call this when user data is updated
    redis_client.delete(f"user:{user_id}")
```

ElastiCache Redis cluster configuration:

```
resource "aws_elasticache_replication_group" "session_store" {
  replication_group_id      = "session-store"
  description                = "Session storage with HA"

  node_type                  = "cache.r7g.large"
  num_cache_clusters        = 3 # 1 primary + 2 replicas

  # Multi-AZ with automatic failover
  multi_az_enabled          = true
  automatic_failover_enabled = true

  # Encryption
```

```

at_rest_encryption_enabled = true
transit_encryption_enabled = true
auth_token                  = var.redis_auth_token

subnet_group_name           = aws_elasticache_subnet_group.main.name
security_group_ids          = [aws_security_group.redis.id]

# Maintenance and backup
maintenance_window          = "sun:05:00-sun:06:00"
snapshot_retention_limit    = 5
snapshot_window             = "04:00-05:00"
}

```

Q25. How do you design a data lake on AWS?

Answer:

AWS data lake reference architecture:

Data Sources

- └─ RDS (transactional data via DMS CDC)
- └─ Kinesis Data Streams (real-time events)
- └─ S3 uploads (batch files, logs)
- └─ Third-party SaaS APIs



S3 Data Lake (organized in zones)

- └─ Raw/Landing Zone (unmodified source data)
s3://company-datalake/raw/source=rds/table=orders/date=2025-03-15/
- └─ Processed/Curated Zone (cleaned, enriched)
s3://company-datalake/processed/orders/year=2025/month=03/day=15/
- └─ Aggregated/Presentation Zone (analytics-ready)
s3://company-datalake/analytics/daily-sales/



AWS Glue (ETL + Data Catalog)

- Crawlers: auto-discover schema, update catalog
- ETL Jobs: PySpark transforms (raw → processed)
- Data Catalog: Hive-compatible metastore



Query & Analytics

- └─ Amazon Athena (SQL on S3; pay per query)
- └─ Amazon Redshift (data warehouse; complex analytics)
- └─ Amazon QuickSight (BI dashboards)

S3 data lake best practices:

```
# Use Parquet format (10x smaller than CSV; 10x faster queries)
# Partition data by date for query performance

# Lake Formation: fine-grained access control on the data catalog
resource "aws_lakeformation_permissions" "analyst_orders" {
  principal = aws_iam_role.data_analyst.arn

  permissions = ["SELECT"] # Read-only

  table {
    database_name = "orders_db"
    name          = "orders"
  }

  # Column-level security: analysts cannot see PII columns
  table_with_columns {
    database_name = "orders_db"
    name          = "customers"
    column_names  = ["customer_id", "order_count", "total_spend"]
    # Excluded: name, email, phone, address
  }
}
```

Q26. How do you implement backup and data protection for critical AWS workloads?

Answer:

AWS Backup — centralized backup management:

```
resource "aws_backup_plan" "production" {
  name = "production-backup-plan"

  rule {
    rule_name          = "daily-backup"
    target_vault_name = aws_backup_vault.main.name

    schedule          = "cron(0 3 * * ? *)" # 3 AM UTC daily
    start_window      = 60 # Minutes to start after schedule
    completion_window = 120 # Minutes to complete

    lifecycle {
      cold_storage_after = 30 # Move to cold storage after 30 days
      delete_after       = 365 # Delete after 1 year
    }
  }
}
```

```

# Copy to DR region
copy_action {
  lifecycle {
    delete_after = 30
  }
  destination_vault_arn = "arn:aws:backup:us-west-
2:${var.account_id}:backup-vault:dr-vault"
}
}

rule {
  rule_name          = "weekly-backup"
  target_vault_name  = aws_backup_vault.main.name
  schedule            = "cron(0 4 ? * SUN *)" # Sunday 4 AM

  lifecycle {
    delete_after = 90
  }
}

# Apply to all production resources by tag
resource "aws_backup_selection" "production" {
  plan_id      = aws_backup_plan.production.id
  name         = "production-resources"
  iam_role_arn = aws_iam_role.backup.arn

  selection_tag {
    type = "STRINGEQUALS"
    key  = "Backup"
    value = "required"
  }
}

```

Section 5: IAM, Security & Compliance

{#section-5}

Q27. Explain IAM roles, policies, and best practices for an enterprise AWS environment.

Answer:

IAM hierarchy:

IAM Policies → Define permissions (what actions on which resources)
IAM Roles → Trusted entity that can assume the role (EC2, Lambda, cross-account)
IAM Users → Human identities (minimize; prefer SSO + roles)
IAM Groups → Collection of users (use for bulk policy attachment)

Policy types:

Type	Scope	Use Case
AWS Managed	AWS-maintained	Quick start; don't customize
Customer Managed	Your account	Custom business requirements
Inline	Embedded in entity	Strict 1:1 binding needed
SCPs	Organization-wide	Guardrails; cannot override
Permission Boundaries	Delegation limit	Allow developers to create roles
Resource-based	Attached to resource	S3 bucket policy; cross-account
Session policies	AssumeRole	Temporary restriction

Least privilege IAM for Lambda:

```
# Lambda role: only the permissions this specific function needs
resource "aws_iam_role_policy" "payment_lambda" {
  name = "payment-lambda-policy"
  role = aws_iam_role.payment_lambda.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Sid    = "DynamoDBAccess"
        Effect = "Allow"
        Action = [
          "dynamodb:GetItem",
          "dynamodb:PutItem",
          "dynamodb:UpdateItem"
          # NOT: dynamodb>DeleteItem, dynamodb:Scan (not needed)
        ]
        Resource = [
          aws_dynamodb_table.orders.arn # Not "*"
        ]
      },
      {
        Sid    = "SecretsAccess"
        Effect = "Allow"
```

```

        Action = ["secretsmanager:GetSecretValue"]
        Resource = [
            "arn:aws:secretsmanager:us-east-1:${var.account_id}:secret:payments/*"
        ]
    },
    {
        Sid      = "KMSDecrypt"
        Effect   = "Allow"
        Action   = ["kms:Decrypt", "kms:GenerateDataKey"]
        Resource = [aws_kms_key.payments.arn]
    }
]
})
}

```

Enterprise IAM best practices:

1. Eliminate IAM users: Use IAM Identity Center (SSO) + corporate IdP (Okta/Azure AD)
2. No long-lived access keys: Use OIDC/roles; if keys needed, rotate every 90 days
3. Permission boundaries: Developers can create roles only within a boundary
4. Condition keys: Restrict by IP, MFA status, time of day
5. IAM Access Analyzer: Continuously detect overly permissive policies
6. CloudTrail: All IAM events logged; alert on sensitive actions (CreateLoginProfile, etc.)

Q28. How do you manage secrets in AWS? Compare options.

Answer:

Options comparison:

Service	Use Case	Cost	Auto-rotation
AWS Secrets Manager	DB passwords, API keys, credentials	0.40/secret/month+ 0.05/10K API calls	 Built-in (Lambda rotation)
SSM Parameter Store Standard	Config values, non-secrets	Free	 Manual

Service	Use Case	Cost	Auto-rotation
SSM Parameter Store SecureString	Encrypted config, lightweight secrets	Free (standard tier)	 Manual
KMS	Encryption keys; encrypt data, not store secrets	1/ <i>CMK/month</i> + 0.03/10K API calls	 Annual key rotation

My guidelines:

- Database credentials → Secrets Manager (auto-rotation pays for itself)
- Application config (non-sensitive) → SSM Parameter Store Standard
- Encrypted application config (semi-sensitive) → SSM SecureString
- API keys for third-party services → Secrets Manager

Secrets Manager auto-rotation setup:

```
resource "aws_secretsmanager_secret" "db_password" {
  name                  = "production/payments/db-password"
  recovery_window_in_days = 30 # Grace period before deletion

  kms_key_id = aws_kms_key.secrets.arn
}

resource "aws_secretsmanager_secret_rotation" "db_password" {
  secret_id          = aws_secretsmanager_secret.db_password.id
  rotation_lambda_arn = aws_lambda_function.rotate_db_secret.arn

  rotation_rules {
    automatically_after_days = 30 # Rotate every 30 days
  }
}
```

Application code — never hardcode, always fetch at runtime:

```
import boto3
import json

def get_db_credentials():
    client = boto3.client('secretsmanager')
    response = client.get_secret_value(
        SecretId='production/payments/db-password'
    )
```

```
secret = json.loads(response['SecretString'])
return secret['username'], secret['password']
```

```
# Cache in memory (refresh before TTL)
# Never log the secret value
# Never pass as command-line argument (visible in ps output)
```

Q29. How do you implement encryption at rest and in transit for AWS services?

Answer:

Encryption at rest:

```
# S3: Server-side encryption with CMK
resource "aws_s3_bucket_server_side_encryption_configuration" "app" {
  bucket = aws_s3_bucket.app.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm      = "aws:kms"
      kms_master_key_id = aws_kms_key.s3.arn
    }
    bucket_key_enabled = true # Reduce KMS API calls by 99% (save cost)
  }
}

# EBS: Encrypted volumes
resource "aws_ebs_encryption_by_default" "default" {
  enabled = true # Account-level: all new EBS volumes encrypted by default
}

# RDS: Encrypted at rest
resource "aws_db_instance" "production" {
  storage_encrypted = true
  kms_key_id        = aws_kms_key.rds.arn
  # Note: encryption can only be set at creation time for RDS
}
```

Encryption in transit:

```
ALB → EC2: Use HTTPS listener; SSL certificate from ACM; terminate at ALB
or end-to-end
EC2 → RDS: require_ssl parameter in RDS parameter group; application uses
ssl=require
API Gateway: HTTPS only; reject HTTP
S3: Bucket policy denies non-HTTPS requests (see Q21)
```

Lambda VPC: Traffic between Lambda and VPC resources uses AWS backbone (encrypted)

KMS CMK key policy best practices:

```
{
  "Statement": [
    {
      "Sid": "Enable account root for key recovery only",
      "Effect": "Allow",
      "Principal": {"AWS": "arn:aws:iam::ACCOUNT_ID:root"},
      "Action": ["kms:*"],
      "Resource": "*"
    },
    {
      "Sid": "Allow key administrators",
      "Effect": "Allow",
      "Principal": {"AWS": "arn:aws:iam::ACCOUNT_ID:role/kms-admin-role"},
      "Action": [
        "kms:Create*", "kms:Describe*", "kms:Enable*", "kms:List*",
        "kms:Put*", "kms:Update*", "kms:Revoke*", "kms:Disable*",
        "kms:Delete*", "kms:ScheduleKeyDeletion", "kms:CancelKeyDeletion"
      ],
      "Resource": "*"
    },
    {
      "Sid": "Allow service usage (NOT key administration)",
      "Effect": "Allow",
      "Principal": {"AWS": "arn:aws:iam::ACCOUNT_ID:role/payments-lambda-
role"},
      "Action": ["kms:Encrypt", "kms:Decrypt", "kms:GenerateDataKey*"],
      "Resource": "*"
    }
  ]
}
```

Q30. How do you implement AWS GuardDuty and Security Hub for threat detection?

Answer:

GuardDuty — ML-based threat detection:

```
# Enable in every region in every account (delegate to security account)
resource "aws_guardduty_detector" "main" {
  enable = true
}
```

```

datasources {
  s3_logs {
    enable = true # Detect malicious S3 activity (data exfiltration)
  }
  kubernetes {
    audit_logs { enable = true } # K8s API anomalies
  }
  malware_protection {
    scan_ec2_instance_with_findings {
      ebs_volumes { enable = true }
    }
  }
}

finding_publishing_frequency = "SIX_HOURS"
}

```

Security Hub — aggregates findings from multiple services:

```

resource "aws_securityhub_account" "main" {}

# Enable standards
resource "aws_securityhub_standards_subscription" "cis" {
  standards_arn = "arn:aws:securityhub::ruleset/cis-aws-foundations-
benchmark/v/1.4.0"
}

resource "aws_securityhub_standards_subscription" "aws_foundational" {
  standards_arn = "arn:aws:securityhub:us-east-1::standards/aws-
foundational-security-best-practices/v/1.0.0"
}

```

Automated remediation for common findings:

```

# Lambda: Auto-remediate "S3 bucket public read access" finding
def handler(event, context):
    # Triggered by EventBridge rule: SecurityHub finding severity = CRITICAL
    finding = event['detail']['findings'][0]

    if finding['Type'] == ['Software and Configuration Checks/AWS Security
Best Practices/S3.2']:
        bucket_name = finding['Resources'][0]['Id'].split(':')[1]

        s3 = boto3.client('s3')
        s3.put_public_access_block(
            Bucket=bucket_name,
            PublicAccessBlockConfiguration={
                'BlockPublicAcls': True,
                'IgnorePublicAcls': True,
                'BlockPublicPolicy': True,

```



```
        'RestrictPublicBuckets': True
    }
)
print(f"Remediated: Blocked public access on {bucket_name}")
```

Q31. How do you implement AWS Config for compliance monitoring?

Answer:

AWS Config records every configuration change to every supported resource.

```
resource "aws_config_configuration_recorder" "main" {
  name      = "default"
  role_arn = aws_iam_role.config.arn

  recording_group {
    all_supported          = true    # Record all resource types
    include_global_resource_types = true # Include IAM resources
  }
}

resource "aws_config_delivery_channel" "main" {
  name            = "default"
  s3_bucket_name = aws_s3_bucket.config.id
  sns_topic_arn  = aws_sns_topic.config_notifications.arn
}
```

Config rules for compliance:

```
# Rule: All EC2 instances must have approved AMIs
resource "aws_config_rule" "approved_amis" {
  name = "approved-amis-by-id"
  source {
    owner          = "AWS"
    source_identifier = "APPROVED_AMIS_BY_ID"
  }
  input_parameters = jsonencode({
    amiIds = "ami-12345,ami-67890"
  })
}

# Rule: S3 buckets must have default encryption
resource "aws_config_rule" "s3_encryption" {
  name = "s3-bucket-server-side-encryption-enabled"
  source {
```

```

    owner          = "AWS"
    source_identifier = "S3_BUCKET_SERVER_SIDE_ENCRYPTION_ENABLED"
  }
}

# Rule: RDS instances must be encrypted
resource "aws_config_rule" "rds_encryption" {
  name = "rds-storage-encrypted"
  source {
    owner          = "AWS"
    source_identifier = "RDS_STORAGE_ENCRYPTED"
  }
}

```

Config Conformance Packs (pre-built compliance packs):

```

# Deploy PCI-DSS conformance pack
aws configservice put-conformance-pack \
  --conformance-pack-name pci-dss \
  --template-s3-uri s3://aws-conformance-packs-templates/Operational-Best-Practices-for-PCI-DSS.yaml

```

Section 6: Infrastructure-as-Code — Terraform & CloudFormation {#section-6}

Q32. How do you structure a Terraform project for AWS infrastructure at scale?

Answer:

Repository structure:

```

terraform/
├── modules/                                # Reusable modules
│   └── vpc/
│       ├── main.tf
│       ├── variables.tf
│       └── outputs.tf

```

```

├── versions.tf
├── eks/
├── rds-aurora/
├── alb/
├── environments/                                # Environment-specific configurations
│   ├── prod/
│   │   ├── main.tf                            # References modules
│   │   ├── variables.tf
│   │   ├── terraform.tfvars                  # Environment values
│   │   └── backend.tf                         # S3 remote state config
│   ├── staging/
│   └── dev/
├── global/                                    # Account-wide resources
│   ├── iam/
│   ├── route53/
│   └── kms/

```

Root module example:

```

# environments/prod/main.tf

module "vpc" {
  source = "../../modules/vpc"

  environment      = "prod"
  vpc_cidr         = "10.0.0.0/16"
  availability_zones = ["us-east-1a", "us-east-1b", "us-east-1c"]
  private_subnets = ["10.0.11.0/24", "10.0.12.0/24", "10.0.13.0/24"]
  public_subnets  = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"]
  database_subnets = ["10.0.21.0/24", "10.0.22.0/24", "10.0.23.0/24"]

  enable_nat_gateway = true
  single_nat_gateway = false # One per AZ for HA
  enable_flow_logs   = true

  tags = local.common_tags
}

module "rds" {
  source = "../../modules/rds-aurora"

  cluster_identifier = "payments-prod"
  engine             = "aurora-postgresql"
  engine_version     = "15.4"
  instance_class     = "db.r7g.xlarge"

  vpc_id      = module.vpc.vpc_id
  subnet_ids  = module.vpc.database_subnet_ids

  tags = local.common_tags
}

```

Remote state with S3 + DynamoDB:

```
# environments/prod/backend.tf
terraform {
  backend "s3" {
    bucket      = "company-tfstate-prod"
    key         = "prod/main.tfstate"
    region     = "us-east-1"
    encrypt     = true
    kms_key_id  = "arn:aws:kms:us-east-1:123:key/xxx"
    dynamodb_table = "terraform-state-lock"
  }
}
```

Q33. How do you handle sensitive values (passwords, keys) in Terraform?

Answer:

Never put secrets in Terraform code. Use these patterns:

Pattern 1: Data source from Secrets Manager:

```
# Fetch secret at plan/apply time – never stored in state in plaintext
data "aws_secretsmanager_secret_version" "db_password" {
  secret_id = "production/payments/db-password"
}

resource "aws_db_instance" "production" {
  password =
    jsondecode(data.aws_secretsmanager_secret_version.db_password.secret_string)
    ["password"]
}
# WARNING: Terraform state will contain the password in plaintext
# Mitigate: use sensitive = true in output + restrict state access
```

Pattern 2: Mark outputs as sensitive:

```
output "db_password" {
  value      = aws_db_instance.production.password
  sensitive = true # Redacted in CLI output
}
```

Pattern 3: Use ignore_changes for auto-rotated secrets:

```
resource "aws_db_instance" "production" {
  password = var.initial_db_password # Only used at creation

  lifecycle {
    ignore_changes = [password] # Secrets Manager owns rotation; don't
drift
  }
}
```

Pattern 4: Never store in tfvars files in git:

```
# .gitignore
*.tfvars          # Contains sensitive values
.terraform/       # Contains provider credentials
terraform.tfstate # Contains resource attributes including secrets
*.tfstate.backup
```

Terraform state security:

```
# S3 state bucket: versioned, encrypted, access-logged
resource "aws_s3_bucket_versioning" "tfstate" {
  bucket = aws_s3_bucket.tfstate.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "tfstate" {
  bucket = aws_s3_bucket.tfstate.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "aws:kms"
      kms_master_key_id = aws_kms_key.tfstate.arn
    }
  }
}
```

```
# Only Terraform execution role can access state
resource "aws_s3_bucket_policy" "tfstate" {
  bucket = aws_s3_bucket.tfstate.id
  policy = jsonencode({
    Statement = [{
      Effect    = "Deny"
      Principal = "*"
      Action    = ["s3:GetObject", "s3:PutObject"]
      Resource  = "${aws_s3_bucket.tfstate.arn}/*"
      Condition = {
        StringNotEquals = {
          "aws:PrincipalArn" = aws_iam_role.terraform_execution.arn

```

```
}
  }
}
})
}
```

Q34. Compare Terraform with AWS CloudFormation and AWS CDK.

Answer:

Feature	Terraform	CloudFormation	AWS CDK
Language	HCL (declarative)	JSON/YAML (declarative)	TypeScript/Python/Java/Go (imperative)
Multi-cloud	✅ Yes	❌ AWS only	❌ AWS only
State management	External (S3+DynamoDB)	Managed by AWS	Managed by AWS (via CFn)
Drift detection	terraform plan	Stack drift detection	Via CDK diff
Rollback	Manual (apply previous)	Automatic rollback on failure	Via CDK deploy rollback
Testing	Terratest, terraform test	taskcat	CDK assertions (native unit testing)
Ecosystem	Terraform Registry (huge)	CloudFormation Registry	CDK Construct Library
Learning curve	Medium (HCL)	Medium (YAML/JSON)	Low for developers (familiar language)
Import existing	terraform import	CloudFormation resource import	cdk import

When I choose each:

Terraform:
- Multi-cloud or future multi-cloud possible

- Team already knows Terraform
- Need rich module ecosystem (Terraform Registry)
- Want consistent approach across AWS + Azure + GCP

CloudFormation:

- Pure AWS; tight integration with AWS services (Service Catalog, Config)
- Need native rollback on stack failure
- Compliance requirement to use AWS-native tools

AWS CDK:

- Developer-led infrastructure (TypeScript/Python devs writing infra)
- Complex conditional logic or loops (CDK uses real programming constructs)
- Building internal construct libraries for teams

Q35. How do you implement a Terraform CI/CD pipeline?

Answer:

Pipeline design:

```
# GitHub Actions: Terraform CI/CD
name: Terraform
on:
  pull_request:
    paths: ['terraform/**']
  push:
    branches: [main]
    paths: ['terraform/**']

permissions:
  id-token: write # OIDC
  contents: read
  pull-requests: write

jobs:
  terraform:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: terraform/environments/prod

    steps:
      - uses: actions/checkout@v4

      - name: Configure AWS credentials via OIDC
        uses: aws-actions/configure-aws-credentials@v4
        with:
```

```
role-to-assume: ${vars.TERRAFORM_ROLE_ARN}
aws-region: us-east-1
```

- ```
- name: Setup Terraform
 uses: hashicorp/setup-terraform@v3
 with:
 terraform_version: "1.8.0"

- name: Terraform Format Check
 run: terraform fmt -check -recursive

- name: Terraform Init
 run: terraform init

- name: Security Scan (Checkov)
 uses: bridgecrewio/checkov-action@master
 with:
 directory: .
 framework: terraform

- name: Terraform Validate
 run: terraform validate

- name: Terraform Plan
 id: plan
 run: terraform plan -out=tfplan -no-color

- name: Post Plan to PR
 if: github.event_name == 'pull_request'
 uses: actions/github-script@v7
 with:
 script: |
 const output = `#### Terraform Plan 
 \\`\\`\\`\\`n$${ steps.plan.outputs.stdout }}\\`\\`\\`\\`;
 github.rest.issues.createComment({
 issue_number: context.issue.number,
 owner: context.repo.owner,
 repo: context.repo.repo,
 body: output
 });

- name: Terraform Apply
 if: github.ref == 'refs/heads/main' && github.event_name == 'push'
 run: terraform apply -auto-approve tfplan
```

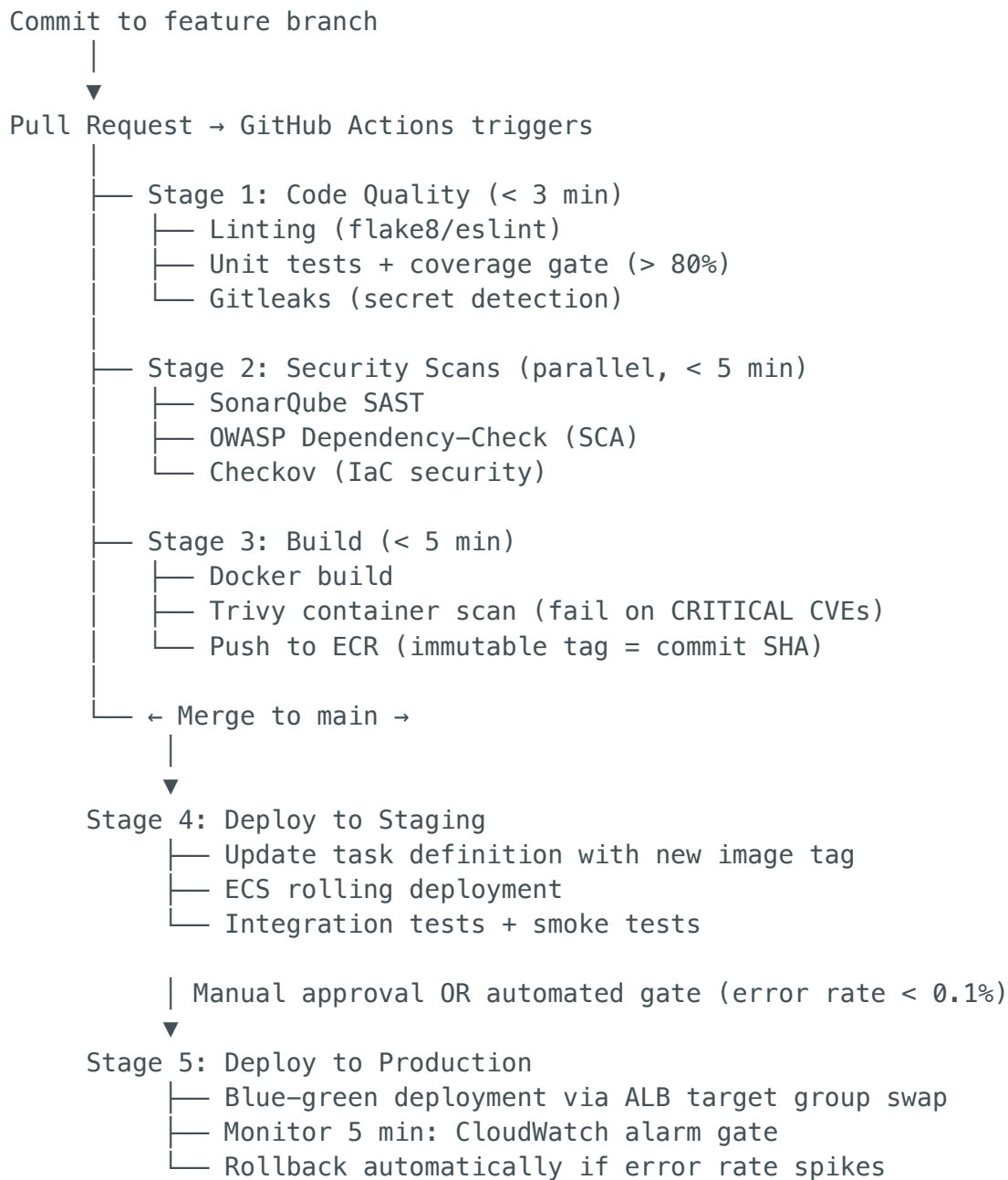
## Section 7: CI/CD, Automation & DevOps Practices {#section-7}



## Q36. Design a CI/CD pipeline for an application deployed to AWS. Include security scanning.

Answer:

Full pipeline with security gates:



Blue-green deployment on ECS:

```
Deploy new version to green task definition
aws ecs update-service \
 --cluster production \
```

```
--service payments-api \
--task-definition payments-api:new-version \
--deployment-configuration "deploymentCircuitBreaker=
{enable=true,rollback=true}" \
--force-new-deployment

ECS deployment circuit breaker: auto-rollback if tasks fail to start
Deployment configuration: minimumHealthyPercent=100, maximumPercent=200
→ Rolling replace (100% healthy maintained throughout)
```

---

## Q37. How do you use AWS CodePipeline and CodeBuild for CI/CD?

**Answer:**

**CodePipeline structure:**

```
resource "aws_codepipeline" "app_pipeline" {
 name = "payments-api-pipeline"
 role_arn = aws_iam_role.codepipeline.arn

 artifact_store {
 location = aws_s3_bucket.pipeline_artifacts.bucket
 type = "S3"
 encryption_key {
 id = aws_kms_key.pipeline.arn
 type = "KMS"
 }
 }
}

stage {
 name = "Source"
 action {
 name = "Source"
 category = "Source"
 owner = "AWS"
 provider = "CodeStarSourceConnection"
 version = "1"
 output_artifacts = ["source_output"]
 configuration = {
 ConnectionArn = aws_codestarconnections_connection.github.arn
 FullRepositoryId = "company/payments-api"
 BranchName = "main"
 DetectChanges = true
 }
 }
}

stage {
```

```

name = "Build"
action {
 name = "Build"
 category = "Build"
 owner = "AWS"
 provider = "CodeBuild"
 version = "1"
 input_artifacts = ["source_output"]
 output_artifacts = ["build_output"]
 configuration = {
 ProjectName = aws_codebuild_project.build.name
 }
}

stage {
 name = "Deploy-Staging"
 action {
 name = "Deploy"
 category = "Deploy"
 owner = "AWS"
 provider = "ECS"
 version = "1"
 input_artifacts = ["build_output"]
 configuration = {
 ClusterName = "staging-cluster"
 ServiceName = "payments-api"
 FileName = "imagedefinitions.json"
 }
 }
}

stage {
 name = "Approve-Production"
 action {
 name = "ManualApproval"
 category = "Approval"
 owner = "AWS"
 provider = "Manual"
 version = "1"
 configuration = {
 NotificationArn = aws_sns_topic.approvals.arn
 CustomData = "Review staging deployment before promoting to
production"
 }
 }
}

```

## CodeBuild buildspec.yml:

```
version: 0.2
```

```
phases:
```

```

pre_build:
 commands:
 - echo Logging into ECR...
 - aws ecr get-login-password --region us-east-1 | docker login --
username AWS --password-stdin $ECR_REGISTRY
 - COMMIT_HASH=$(echo $CODEBUILD_RESOLVED_SOURCE_VERSION | cut -c 1-7)
 - IMAGE_TAG=${COMMIT_HASH:=latest}

build:
 commands:
 - echo Build started on `date`
 - docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG .
 - trivy image --exit-code 1 --severity CRITICAL
$ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG

post_build:
 commands:
 - docker push $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG
 - printf ' [{"name": "payments-api", "imageUri": "%s"}] '
$ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG > imagedefinitions.json

artifacts:
 files:
 - imagedefinitions.json

cache:
 paths:
 - /root/.m2/**/* # Maven cache
 - /root/.gradle/** # Gradle cache

```

## Q38. How do you implement automation scripts for AWS infrastructure provisioning?

**Answer:**

**Python with boto3 — infrastructure automation example:**

```

#!/usr/bin/env python3
"""
Automated EC2 environment provisioner with error handling and idempotency.
"""
import boto3
import json
import time
import logging
from typing import Optional

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

```

```

class EC2Provisioner:
 def __init__(self, region: str = 'us-east-1'):
 self.ec2 = boto3.client('ec2', region_name=region)
 self.ssm = boto3.client('ssm', region_name=region)

 def get_latest_amazon_linux_ami(self) -> str:
 """Get latest Amazon Linux 2023 AMI (idempotent – same result each
call)"""
 response = self.ssm.get_parameter(
 Name='/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-
default-x86_64'
)
 return response['Parameter']['Value']

 def launch_instance(
 self,
 instance_type: str,
 subnet_id: str,
 security_group_ids: list,
 iam_instance_profile: str,
 name: str,
 user_data: Optional[str] = None,
 tags: Optional[dict] = None
) -> dict:
 """Launch EC2 instance with idempotency via Name tag check"""

 # Check if instance with this name already exists (idempotency)
 existing = self.ec2.describe_instances(Filters=[
 {'Name': 'tag:Name', 'Values': [name]},
 {'Name': 'instance-state-name', 'Values': ['running', 'pending',
'stopped']}
])

 if existing['Reservations']:
 instance_id = existing['Reservations'][0]['Instances'][0]
['InstanceId']
 logger.info(f"Instance {name} already exists: {instance_id}")
 return existing['Reservations'][0]['Instances'][0]

 ami_id = self.get_latest_amazon_linux_ami()

 tag_specs = [{'ResourceType': 'instance', 'Tags': [
 {'Key': 'Name', 'Value': name},
 {'Key': 'ManagedBy', 'Value': 'automation-script'},
 *(
 [{'Key': k, 'Value': v} for k, v in (tags or {}).items()]
)
]}]

 launch_params = {
 'ImageId': ami_id,
 'InstanceType': instance_type,
 'SubnetId': subnet_id,
 'SecurityGroupIds': security_group_ids,
 'IamInstanceProfile': {'Name': iam_instance_profile},
 'MetadataOptions': {

```

```

 'HttpTokens': 'required', # IMDSv2 required (security)
 'HttpEndpoint': 'enabled'
 },
 'TagSpecifications': tag_specs,
 'MinCount': 1,
 'MaxCount': 1
}

if user_data:
 import base64
 launch_params['UserData'] =
base64.b64encode(user_data.encode()).decode()

response = self.ec2.run_instances(**launch_params)
instance = response['Instances'][0]

logger.info(f"Launched instance {instance['InstanceId']}")
self._wait_for_instance_running(instance['InstanceId'])

return instance

def _wait_for_instance_running(self, instance_id: str, timeout: int =
300):
 """Wait for instance to reach running state"""
 waiter = self.ec2.get_waiter('instance_running')
 waiter.wait(
 InstanceIds=[instance_id],
 WaiterConfig={'Delay': 10, 'MaxAttempts': timeout // 10}
)
 logger.info(f"Instance {instance_id} is now running")

if __name__ == '__main__':
 provisioner = EC2Provisioner()

 instance = provisioner.launch_instance(
 instance_type='t3.medium',
 subnet_id='subnet-xxx',
 security_group_ids=['sg-xxx'],
 iam_instance_profile='SSMInstanceProfile',
 name='dev-app-server',
 tags={'Environment': 'dev', 'Team': 'platform'}
)
 print(f"Instance ready: {instance['InstanceId']}")

```

## Q39. How do you perform zero-downtime deployments on AWS?

**Answer:**

## Strategy comparison:

| Strategy              | Downtime                  | Cost                  | Rollback Speed               | Complexity |
|-----------------------|---------------------------|-----------------------|------------------------------|------------|
| <b>Rolling update</b> | None (if min healthy > 0) | Low (same capacity)   | Slow (re-deploy old version) | Low        |
| <b>Blue-Green</b>     | None (swap takes seconds) | 2x during deployment  | Fast (swap back)             | Medium     |
| <b>Canary</b>         | None                      | +5-25% extra capacity | Fast (shift traffic)         | High       |

## Blue-green on ALB:

```
def blue_green_deploy(new_target_group_arn: str, alb_arn: str, listener_arn: str):
 """Swap ALB listener to new target group (blue-green)"""
 elbv2 = boto3.client('elbv2')

 # Step 1: Verify new (green) target group is healthy
 response =
 elbv2.describe_target_health(TargetGroupArn=new_target_group_arn)
 healthy_targets = [t for t in response['TargetHealthDescriptions']
 if t['TargetHealth']['State'] == 'healthy']

 if len(healthy_targets) < 2:
 raise Exception(f"Insufficient healthy targets: {len(healthy_targets)}")

 # Step 2: Atomic swap (single API call – near-instant)
 elbv2.modify_listener(
 ListenerArn=listener_arn,
 DefaultActions=[{
 'Type': 'forward',
 'TargetGroupArn': new_target_group_arn
 }]
)

 logger.info(f"Traffic switched to new target group: {new_target_group_arn}")

 # Step 3: Monitor for 5 minutes before decommissioning old
 time.sleep(300)

 # Step 4: CloudWatch check
 cloudwatch = boto3.client('cloudwatch')
 # Verify error rate hasn't spiked before returning success
 return True
```

# Section 8: Cost Optimization & FinOps

## {#section-8}

### Q40. How do you reduce AWS infrastructure costs by 30-40%?

Answer:

Cost optimization levers:

| Lever                                  | Typical Savings              | Effort                        |
|----------------------------------------|------------------------------|-------------------------------|
| Savings Plans (1-year compute)         | 30-36% on EC2/Lambda/Fargate | Low (1-time purchase)         |
| Reserved Instances (RDS, ElastiCache)  | 30-40%                       | Low (1-time purchase)         |
| Spot Instances (batch, fault-tolerant) | 60-70% vs on-demand          | Medium (handle interruptions) |
| Rightsizing (Compute Optimizer)        | 15-25%                       | Medium (test and resize)      |
| S3 Intelligent-Tiering                 | 40-68% on infrequent objects | Low (enable on bucket)        |
| Dev environment auto-shutdown          | 60-70% of dev costs          | Medium (Lambda automation)    |
| NAT Gateway optimization               | 50%+ on NAT costs            | Medium (add VPC endpoints)    |

AWS Compute Optimizer rightsizing workflow:

```
Get rightsizing recommendations
aws compute-optimizer get-ec2-instance-recommendations \
 --account-ids 123456789 \
 --filters name=Finding,values=OVER_PROVISIONED \
```



```
--query 'instanceRecommendations[*].{
 Instance: instanceArn,
 Current: currentInstanceType,
 Recommended: recommendationOptions[0].instanceType,
 Savings: recommendationOptions[0].estimatedMonthlySavings.value
}'
```

## Dev environment auto-shutdown:

```
Lambda: stop dev instances outside business hours (save 65% of dev compute cost)
import boto3

def handler(event, context):
 ec2 = boto3.client('ec2')

 action = event.get('action', 'stop') # 'stop' or 'start'

 instances = ec2.describe_instances(Filters=[
 {'Name': 'tag:Environment', 'Values': ['dev', 'staging']},
 {'Name': 'tag:AutoShutdown', 'Values': ['true']},
 {'Name': 'instance-state-name',
 'Values': ['running'] if action == 'stop' else ['stopped']}
])

 instance_ids = [
 i['InstanceId']
 for r in instances['Reservations']
 for i in r['Instances']
]

 if not instance_ids:
 print(f"No instances to {action}")
 return

 if action == 'stop':
 ec2.stop_instances(InstanceIds=instance_ids)
 else:
 ec2.start_instances(InstanceIds=instance_ids)

 print(f"{action.capitalize()}ped {len(instance_ids)} instances")
```

```
EventBridge scheduler
resource "aws_cloudwatch_event_rule" "stop_dev" {
 name = "stop-dev-instances"
 schedule_expression = "cron(0 20 ? * MON-FRI *)" # 8 PM weekdays UTC
}

resource "aws_cloudwatch_event_rule" "start_dev" {
 name = "start-dev-instances"
```

```
schedule_expression = "cron(0 8 ? * MON-FRI *)" # 8 AM weekdays UTC
}
```

## Q41. How do you implement AWS cost allocation and tagging strategy?

**Answer:**

**Mandatory tag strategy:**

```
Required tags (enforced via SCP/Config rule):
Environment: prod | staging | dev | sandbox
Team: platform | payments | identity | data
Project: checkout | auth | analytics | shared
CostCenter: CC-1001 | CC-1002 | CC-1050
ManagedBy: terraform | console | cloudformation
```

**Enforce via AWS Config:**

```
resource "aws_config_rule" "required_tags" {
 name = "required-tags-check"
 source {
 owner = "AWS"
 source_identifier = "REQUIRED_TAGS"
 }
 input_parameters = jsonencode({
 tag1Key = "Environment"
 tag2Key = "Team"
 tag3Key = "CostCenter"
 })
 # Triggers for all taggable resources
}
```

**Cost allocation with Cost Explorer:**

```
Monthly cost breakdown by team
aws ce get-cost-and-usage \
 --time-period Start=2025-03-01,End=2025-04-01 \
 --granularity MONTHLY \
 --metrics BlendedCost \
 --group-by Type=TAG,Key=Team \
```

```
--query 'ResultsByTime[*].Groups[*].{Team: Keys[0], Cost: Metrics.BlendedCost.Amount}'
```

## Section 9: Reliability, DR & High Availability {#section-9}

### Q42. How do you design a disaster recovery strategy on AWS?

Answer:

DR strategies by RTO/RPO:

| Strategy                        | RTO       | RPO       | Cost Multiplier | How it Works                                             |
|---------------------------------|-----------|-----------|-----------------|----------------------------------------------------------|
| <b>Backup &amp; Restore</b>     | Hours     | Hours     | 1x              | Nightly backups to S3; restore from scratch on failure   |
| <b>Pilot Light</b>              | 15-30 min | Minutes   | 1.1x            | Core services running in DR region; scale up on failover |
| <b>Warm Standby</b>             | 5-15 min  | Seconds   | 1.5x            | Scaled-down DR environment always running                |
| <b>Multi-Site Active-Active</b> | < 1 min   | Near-zero | 2x              | Full traffic in both regions; instant failover           |

Pilot Light implementation:

```
DR region: keep database replica running (pilot light)
resource "aws_rds_cluster" "dr_replica" {
 provider = aws.dr_region
 cluster_identifier = "payments-dr"

 replication_source_identifier = aws_rds_cluster.production.arn

 # Aurora Global DB: typically < 1 second replication lag
 global_cluster_identifier = aws_rds_global_cluster.payments.id
```

```

}

DR region: keep ECS service at 0 desired count (pilot light)
resource "aws_ecs_service" "app_dr" {
 provider = aws.dr_region
 desired_count = 0 # Scale to 10 in failover runbook
 cluster = aws_ecs_cluster.dr.id
 task_definition = aws_ecs_task_definition.app_dr.arn
}

```

## Route 53 health-check based failover:

```

resource "aws_route53_health_check" "primary" {
 fqdn = "api.company.com.us-east-1.internal"
 port = 443
 type = "HTTPS"
 resource_path = "/health"
 failure_threshold = 3 # 3 consecutive failures → unhealthy
 request_interval = 30
}

resource "aws_route53_record" "api_primary" {
 zone_id = aws_route53_zone.main.id
 name = "api.company.com"
 type = "A"

 set_identifier = "primary"
 failover_routing_policy { type = "PRIMARY" }
 health_check_id = aws_route53_health_check.primary.id

 alias {
 name = aws_lb.primary.dns_name
 zone_id = aws_lb.primary.zone_id
 evaluate_target_health = true
 }
}

resource "aws_route53_record" "api_failover" {
 zone_id = aws_route53_zone.main.id
 name = "api.company.com"
 type = "A"

 set_identifier = "failover"
 failover_routing_policy { type = "SECONDARY" }

 alias {
 name = aws_lb.dr.dns_name
 zone_id = aws_lb.dr.zone_id
 evaluate_target_health = true
 }
}

```

# Section 10: Monitoring, Observability & Troubleshooting {#section-10}

---

## Q43. How do you set up comprehensive monitoring for AWS infrastructure?

**Answer:**

**Monitoring stack:**

```
AWS Services → CloudWatch Metrics (native integration)
Application → CloudWatch Logs (structured JSON) + X-Ray (distributed tracing)
Infrastructure → CloudWatch Dashboards (operational view)
Alerting → CloudWatch Alarms → SNS → PagerDuty / Slack
```

```
Enhanced view (optional): Grafana + CloudWatch data source
```

**Key alarms for every production service:**

```
ALB: 5xx error rate alarm
resource "aws_cloudwatch_metric_alarm" "alb_5xx" {
 alarm_name = "alb-high-5xx-rate"
 comparison_operator = "GreaterThanThreshold"
 evaluation_periods = 3
 metric_name = "HTTPCode_Target_5XX_Count"
 namespace = "AWS/ApplicationELB"
 period = 60
 statistic = "Sum"
 threshold = 10 # More than 10 errors per minute
 alarm_description = "ALB target 5xx rate is high"
 alarm_actions = [aws_sns_topic.alerts.arn]
 ok_actions = [aws_sns_topic.alerts.arn]

 dimensions = {
 LoadBalancer = aws_lb.app.arn_suffix
 }
}

RDS: High CPU
resource "aws_cloudwatch_metric_alarm" "rds_cpu" {
 alarm_name = "rds-high-cpu"
 comparison_operator = "GreaterThanThreshold"
```

```

evaluation_periods = 5
metric_name = "CPUUtilization"
namespace = "AWS/RDS"
period = 60
statistic = "Average"
threshold = 80
alarm_actions = [aws_sns_topic.alerts.arn]

dimensions = {
 DBInstanceIdentifier = aws_db_instance.production.identifier
}
}

Lambda: Error rate
resource "aws_cloudwatch_metric_alarm" "lambda_errors" {
 alarm_name = "payments-lambda-errors"
 comparison_operator = "GreaterThanThreshold"
 evaluation_periods = 2
 metric_name = "Errors"
 namespace = "AWS/Lambda"
 period = 60
 statistic = "Sum"
 threshold = 5
 treat_missing_data = "notBreaching" # Silence alarm when function has no
traffic
 alarm_actions = [aws_sns_topic.alerts.arn]

 dimensions = {
 FunctionName = aws_lambda_function.payments.function_name
 }
}

```

## Structured logging with Lambda Powertools:

```

from aws_lambda_powertools import Logger

logger = Logger(service="payments-api", level="INFO")

@logger.inject_lambda_context(log_event=False)
def handler(event, context):
 order_id = event['pathParameters']['orderId']

 # Structured logs – searchable in CloudWatch Logs Insights
 logger.info("Processing payment", extra={
 "orderId": order_id,
 "userId": event['requestContext']['authorizer']['userId'],
 "amount": event['body']['amount']
 })

 # CloudWatch Logs Insights query:
 # fields @timestamp, orderId, userId, amount
 # | filter level = "ERROR"
 # | sort @timestamp desc

```

---

## Q44. How do you use AWS X-Ray for distributed tracing?

**Answer:**

**X-Ray setup for Lambda + API Gateway:**

```
from aws_lambda_powertools import Tracer

tracer = Tracer(service="payments-api")

@tracer.capture_lambda_handler
def handler(event, context):
 return process_payment(event)

@tracer.capture_method # Creates a subsegment for this function
def process_payment(event):
 # This creates an X-Ray subsegment
 tracer.put_annotation(key="orderId", value=event['orderId'])
 tracer.put_metadata(key="request", value=event) # Rich debugging data

 with tracer.provider.in_subsegment("dynamo-lookup") as subsegment:
 result = get_order_from_db(event['orderId'])
 subsegment.put_annotation("dbHit", result is not None)

 return result
```

**X-Ray sampling rules (control cost):**

```
resource "aws_xray_sampling_rule" "payments" {
 rule_name = "payments-api-sampling"
 priority = 1
 version = 1
 reservoir_size = 5 # Minimum 5 requests/second always traced
 fixed_rate = 0.05 # Plus 5% of all other requests
 url_path = "/api/payments/*"
 host = "*"
 http_method = "*"
 service_type = "AWS::Lambda::Function"
 service_name = "payments-api"
 resource_arn = "*"
}
```

---

# Q45. How do you troubleshoot a sudden increase in AWS costs?

Answer:

Cost spike investigation process:

```
Step 1: Identify which service spiked
aws ce get-cost-and-usage \
 --time-period Start=2025-03-10,End=2025-03-15 \
 --granularity DAILY \
 --metrics BlendedCost \
 --group-by Type=DIMENSION,Key=SERVICE \
 --query 'ResultsByTime[*].Groups[*].{Service:Keys[0],
Cost:Metrics.BlendedCost.Amount}' \
 | sort -k2 -rn

Step 2: Drill into the spike service
Example: Data Transfer cost spiked
aws ce get-cost-and-usage \
 --time-period Start=2025-03-13,End=2025-03-14 \
 --granularity HOURLY \
 --metrics UsageQuantity \
 --filter '{"Dimensions":{"Key":"SERVICE","Values":["AWS Data Transfer"]}}' \
 --group-by Type=DIMENSION,Key=USAGE_TYPE
```

Common causes and fixes:

| Cause                                   | Detection                            | Fix                                                                        |
|-----------------------------------------|--------------------------------------|----------------------------------------------------------------------------|
| NAT Gateway data transfer spike         | VPC Flow Logs; NAT GW bytes metric   | Add S3/ECR/SSM VPC endpoints; check for accidental cross-AZ routing        |
| EC2 instances not terminated after test | Cost Explorer tags filter            | Clean up; add auto-cleanup Lambda or instance scheduler                    |
| Lambda runaway execution                | Lambda duration metric; X-Ray traces | Check for infinite retry loops; add DLQ; fix error                         |
| RDS I/O spike                           | CloudWatch IOPS metric               | Enable Aurora I/O Optimized pricing if consistently high; optimize queries |
| S3 request surge                        | S3 request metrics by bucket         | Enable CloudFront CDN; check for misconfigured client retry loops          |



| Cause                                  | Detection                    | Fix                                                                  |
|----------------------------------------|------------------------------|----------------------------------------------------------------------|
| DynamoDB<br>OnDemand capacity<br>spike | DynamoDB<br>consumed RCU/WCU | Switch to provisioned with Auto<br>Scaling if pattern is predictable |

## Section 11: Scenario-Based & Behavioral Questions {#section-11}

**Q46. Scenario: Your production RDS database is at 95% CPU. Walk through how you diagnose and resolve it.**

**Answer:**

**Immediate (< 5 minutes):**

```
-- 1. Connect to RDS via SSM port forwarding (no direct internet access
needed)
-- Check active queries:
SELECT pid, state, wait_event_type, wait_event,
 ROUND(EXTRACT(epoch FROM now() - query_start)) as duration_seconds,
 left(query, 100) as query_snippet
FROM pg_stat_activity
WHERE state != 'idle'
ORDER BY duration_seconds DESC
LIMIT 20;

-- 2. Kill runaway query if safe to do so
SELECT pg_terminate_backend(pid) FROM pg_stat_activity
WHERE pid = <blocking_pid> AND state = 'active';
```

**Short-term mitigation (< 30 minutes):**

```
3. Check if Aurora read replica can absorb read traffic
aws rds describe-db-clusters --db-cluster-identifier payments-prod \
 --query 'DBClusters[*].DBClusterMembers[*].{ID:DBInstanceIdentifier,
IsWriter:IsClusterWriter}'
```

```
4. Promote read replica as temporary measure (application routing change)
Update application connection string to read_endpoint for read queries

5. Add a read replica if none exist
aws rds create-db-instance \
 --db-instance-identifier payments-prod-reader-2 \
 --db-cluster-identifier payments-prod \
 --db-instance-class db.r7g.xlarge \
 --engine aurora-postgresql
```

## Root cause analysis:

```
-- 6. Identify most expensive queries (Performance Insights or
pg_stat_statements)
SELECT query,
 calls,
 ROUND(total_exec_time / calls, 2) as avg_ms,
 ROUND(total_exec_time, 2) as total_ms
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;

-- 7. Explain the most expensive query
EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)
SELECT * FROM orders WHERE customer_id = $1 AND status = 'pending';
-- Look for: Sequential Scan (should be Index Scan), hash join, sort spill
```

## Permanent fix (based on root cause):

| Finding                   | Fix                                                                                            |
|---------------------------|------------------------------------------------------------------------------------------------|
| Missing index             | <pre>CREATE INDEX CONCURRENTLY idx_orders_customer_status ON orders(customer_id, status)</pre> |
| N+1 query pattern         | Fix application code to use JOIN instead of per-row queries                                    |
| Reporting query on OLTP   | Route to read replica; or move to separate analytics DB (Redshift/Aurora serverless)           |
| Connection pool exhausted | Add RDS Proxy or PgBouncer; reduce connection limit per app instance                           |

# Q47. Scenario: You need to migrate a 5TB on-premises database to Aurora PostgreSQL with < 30 minutes downtime. How do you do it?

**Answer:**

**Migration approach — AWS DMS with minimal downtime:**

Phase 1: Setup (Week 1–2)

1. Create Aurora PostgreSQL cluster in VPC
2. Set up AWS Schema Conversion Tool (SCT) – convert Oracle/MySQL schema
3. Test schema compatibility; fix issues in staging
4. Set up DMS replication instance in same VPC as Aurora

Phase 2: Initial load (Week 3)

1. DMS Full Load task: copy all 5TB to Aurora
  - Runs in background; production continues on on-prem DB
  - Takes ~8–12 hours for 5TB (depending on network speed)
  - Use Direct Connect for reliable high-speed transfer

Phase 3: CDC (Change Data Capture) – ongoing

2. DMS CDC task: replicate all changes since Full Load started
  - Applies INSERT/UPDATE/DELETE continuously
  - Lag typically < 5 seconds

Phase 4: Cutover (< 30 minutes)

3. Wait for DMS lag to reach < 1 second
  4. Put on-prem DB in read-only mode (maintenance page for users)
  5. Wait for DMS to drain remaining changes (< 30 seconds)
  6. Stop DMS task
  7. Validate row counts: on-prem vs Aurora
  8. Update application connection string → Aurora endpoint
  9. Start application with Aurora; smoke test
  10. Lift maintenance page
- Total cutover window: ~15–20 minutes

**DMS task configuration:**

```
{
 "TargetMetadata": {
 "TargetSchema": "",
 "SupportLobs": true,
 "FullLobMode": false,
 "LobChunkSize": 64,
 "LimitedSizeLobMode": true,
 "LobMaxSize": 32768
 }
}
```

```

},
"FullLoadSettings": {
 "TargetTablePrepMode": "DO_NOTHING",
 "CreatePkAfterFullLoad": false,
 "StopTaskCachedChangesApplied": false,
 "StopTaskCachedChangesNotApplied": false,
 "MaxFullLoadSubTasks": 8,
 "TransactionConsistencyTimeout": 600,
 "CommitRate": 50000
},
"Logging": {
 "EnableLogging": true,
 "LogComponents": [{ "Id": "SOURCE_UNLOAD", "Severity":
"LOGGER_SEVERITY_DEFAULT" }]
}
}

```

## Q48. Scenario: You're asked to reduce the AWS bill by 30% without impacting availability. What's your plan?

**Answer:**

**90-day cost reduction plan:**

**Week 1-2: Assess and quick wins (\$0 effort savings)**

```

Run AWS Trusted Advisor cost checks
aws support describe-trusted-advisor-check-summaries \
 --check-ids "hjLMh88uM8" "Hs4Ma3G197" "Z4AUBRNSmh"
Returns: idle EC2, underutilized EC2, unassociated EIPs

Run Compute Optimizer
aws compute-optimizer export-ec2-instance-recommendations \
 --s3-destination-config bucket=my-bucket,keyPrefix=co-report

Check unattached EBS volumes (pure waste)
aws ec2 describe-volumes --filters Name=status,Values=available \
 --query 'Volumes[*].{ID:VolumeId, Size:Size, State:State,
Cost:"$0.10*Size"}'

```

**Week 2-4: Quick savings (< 1 week implementation)**

| Action                                                | Estimated Savings     |
|-------------------------------------------------------|-----------------------|
| Delete unattached EBS volumes                         | Varies                |
| Release unassociated Elastic IPs (\$0.005/hr each)    | Small                 |
| Enable S3 Intelligent-Tiering on large buckets        | 20-40% of S3 cost     |
| Add S3/DynamoDB VPC endpoints (eliminate NAT costs)   | 20-50% of NAT cost    |
| Rightsize development instances per Compute Optimizer | 20-30% of dev compute |
| Enable auto-shutdown for dev/staging environments     | 60-70% of dev compute |

## Month 2: Commitment-based discounts

```
Purchase 1-year Compute Savings Plans
Covers EC2, Lambda, Fargate automatically
aws savingsplans purchase-savings-plan \
 --savings-plan-type "Compute" \
 --payment-option "NoUpfront" \
 --term-duration-in-years 1 \
 --commitment 50.00 # $50/hr hourly commitment

RDS Reserved Instances (where Savings Plans don't apply)
aws rds purchase-reserved-db-instances-offering \
 --reserved-db-instances-offering-id xxx \
 --reserved-db-instance-id payments-prod-ri
```

## Month 3: Architecture optimization

| Change                                                        | Savings                        |
|---------------------------------------------------------------|--------------------------------|
| Migrate batch jobs to Spot Instances                          | 60-70% on batch compute        |
| Aurora Serverless v2 for dev/staging (scale to 0 idle)        | 80% on non-prod DB             |
| Lambda vs always-on EC2 for event-driven workloads            | 50-80% on applicable workloads |
| CloudFront for static assets (reduce ALB + S3 transfer costs) | 30-50% on data transfer        |

## Q49. How do you maintain AWS infrastructure documentation?

## Answer:

### Documentation that stays current:

#### 1. Architecture diagrams — draw.io / Lucidchart:

- Update as part of each infrastructure change (same PR as Terraform code)
- Store as code: draw.io XML in Git (diffs are meaningful)

#### 2. Runbooks — Confluence / GitHub Wiki:

- Format: Symptom → Steps → Expected outcome → Escalation
- Tested quarterly (actually run the steps in staging)
- Linked from CloudWatch alarms ("OnAlarmActions" → SNS → Slack with runbook link)

#### 3. Architecture Decision Records (ADRs):

- Stored in `docs/adr/` in the repository
- Template: Context → Options → Decision → Consequences
- Captures WHY decisions were made (invaluable when revisiting later)

#### 4. Self-documenting infrastructure (generated):

```
terraform-docs: auto-generates module documentation
terraform-docs markdown table . > README.md

AWS resource tagging: Description tag on all resources
resource "aws_security_group" "app" {
 tags = {
 Description = "App tier: allows traffic from ALB SG only"
 }
}
```

#### 5. Runbook example:

##### **## Runbook: EC2 Instance High CPU**

**\*\*Alarm:\*\*** ec2-high-cpu-production | **\*\*Severity:\*\*** P2

##### **### Symptoms**

- CloudWatch alarm ec2-high-cpu triggers
- Application response times > 2000ms

##### **### Immediate Steps (< 5 min)**

1. SSH via SSM: ``aws ssm start-session --target <instance-id>``
2. Check top processes: ``top -bn1 | head -20``
3. If runaway process: ``kill -9 <pid>`` (if safe)
4. If load is legitimate: Scale out ASG: ``aws autoscaling set-desired-capacity --auto-scaling-group-name app-asg --desired-capacity <current+2>``

### ### Escalation

Not resolved in 15 min → Page platform-team

### ### Post-Incident

- File JIRA for root cause investigation
- Review if auto-scaling should have triggered first

---

## Q50. How do you handle a failed Terraform apply in production?

Answer:

Failure categories and responses:

**Category 1: Partial apply (some resources created, some failed)**

```
1. Check what was created vs. what failed
terraform show # Show current state
terraform plan # Shows what's still pending

2. Fix the root cause (misconfiguration, permissions, quota)

3. Re-apply (Terraform applies only the delta)
terraform apply

If the failed resource is in a bad state:
terraform state rm aws_resource.failed_resource
Then re-import or re-create manually
```

**Category 2: State lock not released after failure**

```
Check who holds the lock
aws dynamodb get-item \
 --table-name terraform-state-lock \
 --key '{"LockID": {"S": "company-tfstate-prod/main.tfstate"}}'

Force unlock (ONLY after confirming no active apply is running)
terraform force-unlock <LOCK_ID>
```

## Category 3: Resource created but configuration wrong

```
Option A: Fix config and re-apply
Option B: Import existing state and fix
terraform import aws_security_group.app sg-abc123

Option C: Taint resource to force replacement (destroys and recreates)
terraform taint aws_instance.problematic
terraform plan # Review destroy/create plan
terraform apply
```

### Prevention:

```
Always plan first and review carefully
terraform plan -out=prod.tfplan
Have a second person review the plan for production changes

Use -target for surgical changes in emergencies
terraform apply -target=aws_security_group.app

Enable state locking + versioning (never lose state)
Keep 90-day state version history in S3
```

---

## Quick Reference: Hitachi AWS Infrastructure Architect Role

|                    |                                                                    |
|--------------------|--------------------------------------------------------------------|
| Core AWS Services: | EC2, S3, RDS/Aurora, VPC, Lambda, IAM, CloudFront, ALB, ECS/EKS    |
| IaC Tools:         | Terraform (primary), CloudFormation, AWS CDK                       |
| Scripting:         | Python (boto3), Bash, PowerShell                                   |
| CI/CD:             | GitHub Actions, CodePipeline, CodeBuild, CodeDeploy                |
| Monitoring:        | CloudWatch, X-Ray, Trusted Advisor, Health Dashboard               |
| Security:          | IAM least privilege, KMS, Secrets Manager, GuardDuty, Security Hub |
| Cost Optimization: | Savings Plans, Spot, Compute Optimizer, Cost Explorer, tagging     |
| DR Patterns:       | Backup/Restore → Pilot Light → Warm Standby → Active-Active        |
| Key Mindset:       | Automate everything; infrastructure as code; security by default   |